

교과목 3 : 초거대언어모델(LLM)

## 단원 4 : 프롬프트 엔지니어링

DAY3. Tools를 활용한 ReAct 에이전트 구현

# 목 차

|                        |       |    |
|------------------------|-------|----|
| 1. Reasoning Prompt    | ..... | 3  |
| 2. Function Calling    | ..... | 22 |
| 3. Tool System         | ..... | 41 |
| 4. Agent Loop          | ..... | 58 |
| 5. ReAct(Reason + Act) | ..... | 76 |

# 1. Reasoning Prompt

## 왜 단계적 추론이 필요한가 — "AI 는 암산을 시키면 틀린다"

LLM 은 계산기가 아니라 "다음 단어를 예측하는" 모델입니다. 그래서 한 번에 답을 뽐으라고 하면 다단계 계산을 자주 틀립니다. "단계적으로 풀어라" 한마디(Chain-of-Thought)가 정답률을 크게 끌어올립니다.

### 1. 현장의 문제

승승장구물 상담 중에는 계산이 자주 나옵니다.

- "이어버드 3개에 10% 쿠폰이면 얼마예요?"
- "할인하고 배송비 더하면 총액이?"

그런데 LLM에게 곧바로 "답만 말해"라고 하면 의외로 자주 틀립니다.

```
ask_direct("이어버드를 79000원에 3개 샀는데 10% 쿠폰을 받았습니다. 총 결제액은?")  
# → "213000" ← 틀림! (정답은 213300)
```

### 2. 왜 틀릴까?

1장에서 배웠듯, LLM은 "다음에 올 가장 그럴듯한 단어(숫자)를 예측" 하는 기계입니다. 진짜로 곱셈·뺄셈을 하는 게 아닙니다.

한 번에 최종 숫자를 뽐으라고 하면, 중간 과정을 건너뛰고 "그럴듯한 숫자"를 찍습니다. 그래서 213300 이 정답인데 213000 처럼 비슷한 숫자를 내놓는 거죠.

비유: 사람한테도 "79000 곱하기 3 곱하기 0.9를 암산으로 한 번에 말해"라고 하면 틀리기 쉽습니다. 하지만 종이에 한 줄씩 적으면 정확해집니다. LLM도 똑같습니다.

### 3. 해결책 — "단계적으로 풀어라"

해결책은 놀랄 만큼 단순합니다. 프롬프트에 "단계적으로 풀어라" 한 줄을 더하는 것.

```
ask_cot("이어버드를 79000원에 3개 샀는데 10% 쿠폰을 받았습니다. 총 결제액은?")
# —
# 1) 상품 합계:  $79000 \times 3 = 237000$ 
# 2) 쿠폰 할인:  $237000 \times 10\% = 23700$ 
# 3) 결제액:  $237000 - 23700 = 213300$ 
# 정답: 213300 ← 맞음!
```

중간 풀이를 글로 쓰게 했더니 정답이 나왔습니다. 이게 **Chain-of-Thought**(생각의 사슬, 줄여서 **CoT**) 입니다.

### 4. 왜 글로 쓰면 정확해지나

핵심 원리가 있습니다. LLM은 출력 토큰을 하나씩 만들면서, 앞서 쓴 자기 출력을 다시 입력으로 참고합니다.

즉, 중간 풀이를 글로 쓰게 하면 그 풀이가 다음 토큰을 만들 때의 "작업 공간(근거)" 이 됩니다. 237000 이라고 써 놓으면, 다음 줄에서 그 숫자를 보고 계산을 이어 갈 수 있는 거죠. 암산이 필산이 되는 셈입니다.

- **직접 답변:** 문제 → "213000" — 중간 작업 공간이 없어 사실상 찍기.
- **CoT:** 문제 → 237000 → 23700 → 213300 → "정답: 213300" — 한 줄이 다음 줄의 근거가 되어 정확.

## 5. 이 강에서 할 일

1. 직접 답변 vs CoT를 같은 8문제로 풀어 정답률을 비교합니다(보통 63% → 100%).
2. CoT 답을 한 번 더 검산하는 자기검증을 배웁니다.
3. CoT가 효과 없거나 오히려 해로운 경우(단순 질문, 함정 문제)도 봅니다 — CoT는 만능이 아닙니다.

미리 알아둘 점: 7강에서 만들 에이전트의 "판단(Thought)" 단계가 바로 이 CoT의 응용입니다. 에이전트가 "다음에 뭘 할지 생각"하게 만드는 게 곧 추론이거든요.

### 핵심 정리

- LLM은 계산기가 아니라 **단어 예측기**라, 암산(직접 답변)을 자주 틀린다.
- "**단계적으로 풀어라**"(CoT)가 중간 풀이를 작업 공간으로 만들어 정답률을 올린다.
- 원리: 모델은 자기가 쓴 **앞 출력을 다음 출력의 근거**로 삼는다.

## CoT vs 직접 답변

직접 답변은 중간 과정 없이 답만, CoT는 풀이를 한 줄씩 쓴 뒤 답을 냅니다. CoT는 토큰을 더 쓰지만 계산·다단계 문제의 정답률이 훨씬 높습니다.

### 1. 두 방식 비교표

| 구분    | 직접 답변         | Chain-of-Thought        |
|-------|---------------|-------------------------|
| 프롬프트  | "답(숫자)만 말해"   | "단계적으로 풀이를 쓴 뒤 마지막에 답해" |
| 토큰/속도 | 적음 / 빠름       | 많음 / 느림                 |
| 정답률   | 낮음(특히 계산·다단계) | 높음                      |
| 쓰는 곳  | 단순 사실 질의      | 계산·추론·다단계 판단            |

핵심 트레이드오프는 "비용·속도 ↔ 정확도"입니다. CoT는 풀이를 쓰느라 토큰(=비용)과 시간을 더 쓰지만, 그 대가로 정확도를 얻습니다.

### 2. 직접 답변 함수

중간 과정을 막고 최종 숫자만 시킵니다.

```
from common import get_genai_client, GEMINI_MODEL
from google.genai import types

client = get_genai_client()

def ask_direct(question: str) -> str:
    """직접 답변: 중간 과정 없이 최종 숫자만 시킨다 → 다단계 계산에서 자주 틀림."""
    resp = client.models.generate_content(
        model=GEMINI_MODEL,
        contents=f"{question}\n설명 없이 최종 숫자(원)만 답하라.",
        config=types.GenerateContentConfig(temperature=0),
    )
    return resp.text
```

"설명 없이 최종 숫자만"이라는 지시가 모델의 손발을 묶습니다. 작업 공간이 없으니 찍을 수 밖에요.

### 3. CoT 함수

반대로 한 줄씩 풀이를 쓰게 하고, 마지막 줄에 정답 형식을 정해 줍니다.

```
def ask_cot(question: str) -> str:
    """CoT: 한 줄씩 풀이를 쓰게 한다.

    [왜] 모델은 앞서 쓴 자기 출력을 다시 입력으로 참고한다. 풀이를 글로 쓰게 하면
    그 풀이가 다음 토큰 생성의 '작업 공간(근거)'이 되어 마지막 답이 정확해진다.
    """
    resp = client.models.generate_content(
        model=GEMINI_MODEL,
        contents=(
            f"{question}\n"
            "단계적으로 풀어라. 각 계산을 한 줄씩 쓰고, "
            "맨 마지막 줄에 '정답: <숫자>' 형식으로 답하라."
        ),
        config=types.GenerateContentConfig(temperature=0),
    )
    return resp.text
```

두 가지 장치가 있습니다.

- "단계적으로 풀어라": 작업 공간(풀이)을 만들게 한다.
- "마지막 줄에 '정답: <숫자>' 형식": 답을 일정한 위치·형식으로 고정해 추출하기 쉽게 한다.

## 4. 답에서 숫자만 뽑아내기

CoT 답변은 풀이가 길어서, 그 안에서 최종 숫자를 뽑아야 채점할 수 있습니다. 정규식으로 "마지막 숫자"를 정답으로 뽑니다.

```
import re

def extract_number(text: str):
    """답변에서 마지막 숫자를 정수로 반환(coma 제거)."""
    nums = re.findall(r"-?\d[\d,]*", text.replace(" ", ""))
    if not nums:
        return None
    return int(nums[-1].replace(",", ""))
```

`re.findall`로 모든 숫자를 찾고, 마지막 것(`nums[-1]`)을 답으로 씁니다. "정답: 213300"이 맨 끝에 오게 프롬프트로 유도했기 때문이죠.

`1,000` 같은 콤마는 `.replace(",", "")`로 제거해 `int()`로 변환합니다.

## 5. 한 문제로 차이 보기

```
q = "이어버드를 79000원에 3개 샀는데 10% 쿠폰을 받았습니다. 총 결제액은?"
print("[직접] ", ask_direct(q))
print("[CoT]\n", ask_cot(q))
```

### 예상 출력

[직접] 213000      ← 한 번에 답 → 틀림

[CoT]

1) 상품 합계:  $79000 \times 3 = 237000$

2) 쿠폰 할인:  $237000 \times 10\% = 23700$

3) 결제액:  $237000 - 23700 = 213300$

정답: 213300      ← 단계를 거쳐 정확

같은 모델, 같은 문제인데 프롬프트 한 줄 차이로 정답이 같렸습니다. 다음 절(자기검증) 후, 04번에서 8문제 전체 정답률을 비교합니다.



## 자기검증(검산)

CoT 로 푼 답도 가끔 틀립니다. "위 풀이가 맞는지 다시 계산해 보라" 고 한 번 더 시키면(self-consistency 응용), 모델이 스스로 오류를 잡아 신뢰도가 올라갑니다. 돈·재고처럼 틀리면 안 되는 계산의 보험입니다.

### 1. CoT도 완벽하지 않다

CoT는 정답률을 크게 높이지만 100%는 아닙니다. 특히 단계가 많거나 숫자가 큰 문제에서는 여전히 틀릴 수 있습니다. 그럼 어떻게 더 믿음직하게 만들까요?

답은 사람이 시험 볼 때 하는 것과 같습니다 — 검산입니다.

비유: 수학 시험에서 답을 다 푼 뒤 "다시 한번 계산해 보는" 그 과정입니다. 한 번 더 보면 실수를 잡을 확률이 올라갑니다.

### 2. self-consistency란?

원래 **self-consistency**(자기일관성) 기법은 이렇습니다.

- 같은 문제를 여러 번 풀게 한다 (temperature를 약간 높여 다양한 풀이 유도).
- 나온 답들 중 가장 많이 나온 답(다수결) 을 채택한다.

여러 번 풀어도 일관되게 같은 답이 나오면, 그 답을 믿을 수 있다는 아이디어죠. 다만 여러 번 호출하면 비용이 배로 듭니다.

그래서 이 강에서는 더 가벼운 변형인 "검산 한 번" 으로 핵심을 체험합니다.

### 3. 검산 함수

CoT로 푼 답을 모델에게 다시 검산시킵니다.

```
from common import get_genai_client, GEMINI_MODEL
from google.genai import types

client = get_genai_client()

def verify(question: str, cot_answer: str) -> str:
    """제출한 풀이를 모델에게 다시 검산시켜 신뢰도를 높인다."""
    resp = client.models.generate_content(
        model=GEMINI_MODEL,
        contents=(
            f"[문제] {question}\n[제출한 풀이]\n{cot_answer}\n\n"
            "위 풀이가 맞는지 다시 계산해 검산하라. "
            "틀렸다면 올바른 값을, 맞다면 그대로 '정답: <숫자>'로 확정하라."
        ),
        config=types.GenerateContentConfig(temperature=0),
    )
    return resp.text
```

핵심은 **이전 풀이를 통째로 보여 주고** "이게 맞는지 다시 계산하라"고 시키는 것입니다. 모델은 그 풀이를 검토하며 오류가 있으면 바로잡습니다.

## 4. 실행

```
q = "이어버드를 79000원에 3개 샀는데 10% 쿠폰을 받았습니다. 총 결제액은?"  
first = ask_cot(q)          # 1차 풀이  
print("[검산 결과]\n", verify(q, first))  # 2차 검산
```

### 예상 출력

```
[검산 결과]  
제출한 풀이를 검토합니다.  
79000 × 3 = 237000 (맞음)  
237000 × 0.1 = 23700 (맞음)  
237000 - 23700 = 213300 (맞음)  
정답: 213300
```

검산을 통해 답이 확정되었습니다. 만약 1차 풀이에 오류가 있었다면, 이 단계에서 바로잡혔을 것입니다.

## 5. 언제 검산을 쓰나

검산은 토큰(비용)을 약 2배로 늘립니다. 그래서 항상 쓰는 게 아니라, 틀리면 큰일 나는 계산에 선택적으로 씁니다.

-  환불 금액 계산, 재고 발주량, 정산 금액 → 검산 필수
-  "오늘 날씨 어때?" 같은 단순 질의 → 검산 불필요(낭비)

현업 에이전트는 "민감한 계산엔 검산 한 번"을 기본 정책으로 두기도 합니다. 비용을 조금 더 내고 사고를 예방하는 거죠. 보험과 같은 개념입니다.

### 핵심 정리

- CoT 답도 틀릴 수 있어, **검산**으로 신뢰도를 높인다.
- `verify()`는 **이전 풀이를 보여 주고 "다시 계산하라"** 고 시킨다.
- 검산은 비용이 2 배지만, **틀리면 안 되는 계산**에서는 값진 보험이다.

## 따라하기 — CoT 정답률 비교

### 목표

쇼핑 계산 문제 8개를 직접 답변과 CoT 두 방식으로 풀어, 정답률을 숫자로 비교합니다. "단계적으로 풀어나가" 한 줄의 효과를 눈으로 확인합니다. 실제 파일은 `code/ch04_cot.py` 입니다.

### 0. 실습 준비 (설치)

```
conda activate agentic

# 4강에 필요한 라이브러리 (대부분 앞 강에서 설치했다면 생략 가능)
# - pandas : 계산 문제 CSV를 다루기 위해
pip install google-genai python-dotenv pandas
```

이번 강은 `data/math_word_problems.csv` (쇼핑 계산 문제 8개 + 정답 `answer`)를 씁니다.

### 1. 공통 시작부 + 데이터 로드

```
# 실행: python code/ch04_cot.py
import sys, pathlib, re
sys.path.append(str(pathlib.Path(__file__).resolve().parent))
from common import get_genai_client, GEMINI_MODEL, DATA
from google.genai import types
import pandas as pd

client = get_genai_client()
# math_word_problems.csv = 쇼핑 계산 문제 8개. answer 컬럼이 정수 정답.
df = pd.read_csv(DATA / "math_word_problems.csv")
print("문제 수:", len(df))
print(df.iloc[0]["question"], "→ 정답:", df.iloc[0]["answer"])
```

## 2. 세 함수 준비

02번에서 만든 `ask_direct`, `ask_cot`, `extract_number`를 그대로 씁니다.

```
def extract_number(text: str):
    """답변에서 마지막 숫자를 정수로 반환(콤마 제거)."""
    nums = re.findall(r"-?\d[\d,]*", text.replace(" ", ""))
    return int(nums[-1].replace(",", "")) if nums else None

def ask_direct(question: str) -> str:
    resp = client.models.generate_content(
        model=GEMINI_MODEL,
        contents=f"{question}\n설명 없이 최종 숫자(원)만 답하라.",
        config=types.GenerateContentConfig(temperature=0),
    )
    return resp.text

def ask_cot(question: str) -> str:
    resp = client.models.generate_content(
        model=GEMINI_MODEL,
        contents=(f"{question}\n단계적으로 풀어라. 각 계산을 한 줄씩 쓰고, "
                  "맨 마지막 줄에 '정답: <숫자>' 형식으로 답하라."),
        config=types.GenerateContentConfig(temperature=0),
    )
    return resp.text
```

### 3. 8문제 전체 정답률 비교

이게 이 강의 핵심 실험입니다. 8문제를 두 방식으로 풀고, 정답(`answer`)과 비교해 각각 몇 개 맞았는지 셉니다.

```
direct_ok = cot_ok = 0
for _, row in df.iterrows():
    ans = int(row["answer"])
    d = extract_number(ask_direct(row["question"])) # 직접 답변의 숫자
    c = extract_number(ask_cot(row["question"]))    # CoT 답변의 숫자
    direct_ok += (d == ans) # bool(True/False)은 1/0으로 더해진다
    cot_ok += (c == ans)
    print(f"{row['problem_id']} 정답={ans}>7} | 직접={d} {'0' if d==ans else 'X'}"
          f" | CoT={c} {'0' if c==ans else 'X'}")

n = len(df)
print(f"\n직접 답변 정답률 : {direct_ok}/{n} = {direct_ok/n:.0%}")
print(f"CoT 정답률      : {cot_ok}/{n} = {cot_ok/n:.0%}")
```

#### 예상 출력

```
M01 정답= 213300 | 직접=213000 X | CoT=213300 0
M02 정답= 386650 | 직접=386650 0 | CoT=386650 0
M03 정답= 45000 | 직접=44000 X | CoT=45000 0
...
직접 답변 정답률 : 5/8 = 63%
CoT 정답률      : 8/8 = 100%
```

파이썬 팁: `direct_ok += (d == ans)` 에서 `(d == ans)` 는 `True / False` 인데, 파이썬은 이를 `1 / 0` 으로 취급해 더합니다. 그래서 "맞은 개수"가 자동으로 쌓입니다.

### 4. 결과 해석

같은 모델, 같은 문제인데 프롬프트 한 줄("단계적으로 풀어라") 로 정답률이 **63% → 100%** 로 뛰었습니다.

- 직접 답변이 틀린 문제(M01, M03 ...)는 모두 다단계 계산입니다.
- M02처럼 단순한 건 직접 답변도 맞습니다.

즉, 계산 단계가 많을수록 **CoT**의 우위가 뚜렷합니다. (모델 버전에 따라 수치는 달라질 수 있지만 경향은 동일합니다.)

## 따라하기 — 자기검증(검산) 추가하기

### 목표

04번에서 만든 CoT 풀이에 검산 단계를 더해, 답의 신뢰도를 한 단계 높입니다. 실제 파일은 `code/ch04_cot.py`의 `verify()` 부분입니다.

### 1. 검산 함수 (03번 복습)

CoT로 푼 답을 모델에게 다시 검토시킵니다.

```
from common import get_genai_client, GEMINI_MODEL
from google.genai import types

client = get_genai_client()

def verify(question: str, cot_answer: str) -> str:
    """제출한 풀이를 모델에게 다시 검산시켜 신뢰도를 높인다."""
    resp = client.models.generate_content(
        model=GEMINI_MODEL,
        contents=(
            f"[문제] {question}\n[제출한 풀이]\n{cot_answer}\n\n"
            "위 풀이가 맞는지 다시 계산해 검산하라. "
            "틀렸다면 올바른 값을, 맞다면 그대로 '정답: <숫자>'로 확정하라."
        ),
        config=types.GenerateContentConfig(temperature=0),
    )
    return resp.text
```

## 2. "풀이 → 검산" 2단계 실행

```
import pandas as pd
df = pd.read_csv(DATA / "math_word_problems.csv")

q = df.iloc[0]["question"]

# 1단계: CoT로 푼다
first = ask_cot(q)
print("[1차 풀이]\n", first)

# 2단계: 그 풀이를 검산한다
print("\n[검산 결과]\n", verify(q, first))
```

### 예상 출력

```
[1차 풀이]
1)  $79000 \times 3 = 237000$ 
2)  $237000 \times 10\% = 23700$ 
3)  $237000 - 23700 = 213300$ 
정답: 213300

[검산 결과]
제출한 풀이를 검토합니다.
 $79000 \times 3 = 237000$  ✓
 $237000 \times 0.1 = 23700$  ✓
 $237000 - 23700 = 213300$  ✓
정답: 213300
```

검산을 거쳐 답이 확정되었습니다. 1차 풀이에 오류가 있었다면 여기서 바로잡혔겠죠.



### 3. 검산이 오류를 잡는 예 (개념)

만약 1차 풀이가 틀렸다고 가정해 봅시다.

```
wrong_solution = """1) 79000 × 3 = 237000
2) 237000 × 10% = 23700
3) 237000 - 23700 = 213000 ← 계산 실수!
정답: 213000"""

print(verify(q, wrong_solution))
```

#### 예상 출력

```
검토 결과, 3단계에 오류가 있습니다.
237000 - 23700 = 213300 이어야 합니다.
정답: 213300 ← 검산이 실수를 바로잡음
```

검산 단계가 틀린 답을 고쳐 줍니다. 이것이 자기검증의 가치입니다.

### 4. 비용을 고려한 사용 전략

검산은 호출을 한 번 더 하므로 토큰(비용)이 약 2배입니다. 그래서 모든 질문에 쓰지 않고, 전략적으로 씁니다.

```
def answer_with_optional_verify(question: str, critical: bool) -> str:
    """critical=True(돈·재고 등 민감)일 때만 검산을 추가한다."""
    first = ask_cot(question)
    if critical:
        return verify(question, first) # 민감한 계산 → 검산
    return first                       # 일반 질문 → 그대로
```

"이 계산이 틀리면 큰일 나는가?"를 기준으로 `critical` 을 정하면 됩니다. 환불액·발주량 같은 건 `True` , 단순 안내는 `False` .

#### 핵심 정리

- "CoT 풀이 → 검산"의 2 단계로 답을 확정한다.
- 검산은 1차 풀이의 계산 실수를 잡아낸다.
- 비용이 2 배이므로 민감한 계산에만 선택적으로 적용한다.

## CoT 가 효과 없는 경우

CoT 는 강력하지만 **만능이 아닙니다**. (a) 단순 사실/상식 질문에는 토큰만 낭비하고, (b) 함정 문제에서는 잘못된 중간 단계가 오히려 오답을 부를 수 있습니다. "**언제 CoT 를 끄는가**" 도 알아야 합니다.

### 1. CoT의 두 가지 함정

앞에서 CoT가 정답률을 63% → 100%로 올리는 걸 봤습니다. 그래서 "무조건 CoT를 켜자"고 생각하기 쉽지만, 두 가지 경우엔 손해입니다.

- (a) 단순 질문: "대한민국 수도는?" 같은 건 직접 답변도 맞습니다. CoT는 토큰(비용)만 더 씁니다.
- (b) 함정 문제: 직관적인 중간 계산이 함정인 문제에서는, CoT가 그 함정에 빠져 오답을 낼 수 있습니다.

## 2. (a) 단순 질문 — 토큰 낭비

`usage_metadata`로 토큰을 비교하면 낭비가 보입니다.

```
from common import get_genai_client, GEMINI_MODEL
from google.genai import types

client = get_genai_client()

def ask_direct(question: str):
    resp = client.models.generate_content(
        model=GEMINI_MODEL,
        contents=f"{question}\n설명 없이 정답만 한 단어로 답하라.",
        config=types.GenerateContentConfig(temperature=0),
    )
    return resp.text.strip(), resp.usage_metadata.total_token_count

def ask_cot(question: str):
    resp = client.models.generate_content(
        model=GEMINI_MODEL,
        contents=f"{question}\n단계적으로 풀어라. 마지막 줄에 '정답: <값>'.",
        config=types.GenerateContentConfig(temperature=0),
    )
    return resp.text.strip(), resp.usage_metadata.total_token_count

d_txt, d_tok = ask_direct("대한민국의 수도는 어디인가?")
c_txt, c_tok = ask_cot("대한민국의 수도는 어디인가?")
print(f"직접: {d_txt} (토큰 {d_tok})")
print(f"CoT : 토큰 {c_tok} ← 같은 정답인데 토큰만 더 씀")
```

### 예상 출력

```
직접: 서울 (토큰 8)
CoT : 토큰 45 ← 같은 정답인데 토큰만 더 씀
```

둘 다 "서울"로 정답이지만, CoT는 토큰을 **5배 넘게** 썼습니다. 단순 질문에는 직접 답변이 빠르고 씁니다.

### 3. (b) 함정 문제 — CoT도 흔들린다

유명한 함정 문제가 있습니다.

야구 배트와 공이 합쳐서 1,100원이다. 배트는 공보다 1,000원 비싸다. 공은 얼마인가?

많은 사람이 직관적으로 "100원!"이라 답하지만, 정답은 **50원**입니다. (공 50 + 배트 1,050 = 1,100, 차이 1,000 ✓)

```
cases = [
    ("배트와 공이 합쳐서 1,100원. 배트는 공보다 1,000원 비싸다. 공은?", "50"),
    ("5대 기계가 5개를 5분에 만든다. 100대가 100개를 만드는 데 몇 분?", "5"), # 정답 5분
]

for q, gold in cases:
    d_txt, _ = ask_direct(q)
    c_txt, _ = ask_cot(q)
    print(f"Q: {q}")
    print(f"  정답: {gold} / 직접: {d_txt[:20]} / CoT 마지막줄: {c_txt.splitlines()[-1]}")
```

이런 문제는 CoT를 써도, 모델이 직관적인(틀린) 중간 단계를 밟으면 그대로 오답으로 이어질 수 있습니다. CoT가 "생각을 글로 쓴다"고 해서 그 생각이 항상 옳은 건 아니니까요.

핵심: CoT는 계산을 정리해 주지만, 추론 방향이 틀리면 정리된 오답이 나옵니다. 이런 문제엔 검산(03번)이나 다른 접근이 더 필요합니다.

## 4. 토큰 총량 비교

```
# (전체 케이스의 토큰을 합산)
print(f"총 토큰 — 직접: {direct_tok_sum} / CoT: {cot_tok_sum} "
      f"(CoT가 {cot_tok_sum - direct_tok_sum}토큰 더 사용)")
```

### 예상 출력

총 토큰 — 직접: 60 / CoT: 320 (CoT가 260토큰 더 사용)

CoT는 정확도를 위해 토큰을 몇 배 더 씁니다. 대량 처리(수만 건)에서는 이 차이가 비용으로 직결됩니다.

## 5. 그래서, 언제 CoT를 쓰나

| 상황              | CoT?     | 이유            |
|-----------------|----------|---------------|
| 다단계 계산·논리       | ✅ 켜다     | 정답을 크게 향상     |
| 단순 사실·상식·록업     | ❌ 끄다     | 추론 불필요, 토큰 낭비 |
| 지연·비용 민감한 대량 처리 | ⚠️ 신중    | 토큰이 몇 배로 늘어남  |
| 함정·역설 문제        | ⚠️ 검산 병행 | CoT도 틀릴 수 있음  |

정리하면: **CoT**는 "다단계 계산·논리"일 때만 켜는 것이 비용 대비 효과가 큼니다. 무조건 켜는 게 아니라, 문제의 성격을 보고 켜고 끄는 판단이 중요합니다.

## 핵심 정리

- CoT 는 만능이 아니다 — 단순 질문엔 토큰 낭비, 함정 문제엔 오답 위험.
- 단순 질문은 직접 답변이 빠르고 싸다.
- 함정 문제는 CoT + 검산을 병행한다.
- CoT 는 다단계 계산·논리일 때만 켜는 것이 비용 대비 효과적이다.

## 2. Function Calling

### 왜 함수 호출이 필요한가 — "두뇌에 손발을 달다"

1장에서 LLM의 두뇌를 잘 쓰는 법(프롬프트·추론)을 배웠습니다. 하지만 두뇌만으로는 "내 주문 어디 왔어?", "재고 있어?"에 답할 수 없습니다. 이제 두뇌에 **손발(도구)**을 달아, LLM이 실제 데이터를 조회하게 만듭니다. 이게 **Function Calling**이고, 에이전트로 가는 **핵심 전환점**입니다.

#### 1. 두뇌만으로는 안 되는 일

1장의 그 문제를 떠올려 봅시다.

```
ask("승승장구물 이어버드 지금 재고 있어요?")  
# → "죄송하지만 실시간 재고는 확인할 수 없습니다 ..." (또는 환각)
```

아무리 프롬프트를 잘 짜고(3강) 단계적으로 생각하게 해도(4강), **재고 데이터 자체에 접근할 수단이 없**으면 답할 수 없습니다. 두뇌는 똑똑하지만 **손발이 없기** 때문입니다.

#### 2. 도구(Tool)를 쥐여 주면?

여기서 도구가 등장합니다. 도구는 거창한 게 아니라, 그냥 평범한 파이썬 함수입니다.

```
def get_stock(product_name: str) -> str:  
    """상품명을 받아 현재 재고 수량을 반환한다."""  
    # inventory.csv를 조회해서...  
    return "승승 무선 블루투스 이어버드 SE 재고 225개"
```

이 함수를 LLM에게 "쓸 수 있는 도구"로 쥐여 주면, LLM은 재고 질문이 왔을 때 스스로 이 함수를 호출해서 진짜 데이터를 가져옵니다.

```
question = "이어버드 재고 있어? 그리고 달러로 환산하려면 환율이 얼마야?"  
# → LLM이 get_stock 도구와 get_exchange_rate 도구를 둘 다 호출하고,  
# 그 결과로 정확한 답을 만든다.
```

비유: 1~4강에서 우리는 똑똑한 직원(LLM)을 채용했습니다. 하지만 그 직원에게 **창고 열쇠(재고 도구)**와 **환율 단말기(환율 도구)**를 주지 않으면, 아무리 똑똑해도 "창고에 가서 확인"할 수 없습니다. Function Calling이 바로 그 열쇠와 단말기를 쥐여 주는 일입니다.

### 3. 이 강에서 배울 것

| 목차    | 내용                               |
|-------|----------------------------------|
| 02    | LLM은 함수를 직접 실행하지 못한다 (가장 중요한 개념) |
| 03    | 자동 실행 vs 수동 루프의 차이               |
| 04~06 | 도구 함수 만들기 → 자동 호출 → 수동 루프로 결정 관찰 |
| 07    | 도구가 실패할 때 우아하게 처리하기              |

### 4. 이 강에서 만들 도구

승승장구물 운영에 필요한 두 가지 도구를 만듭니다.

- `get_stock(product_name)`: 재고 조회 ( `data/inventory.csv` )
- `get_exchange_rate(currency)`: 환율 조회 ( `data/exchange_rates.csv` )

이 두 도구를 LLM에 연결하면, "이어버드 재고를 달러로 환산하면?" 같은 복합 질문도 LLM이 알아서 두 도구를 조합해 답합니다.

## LLM 은 함수를 직접 실행하지 못한다

가장 중요한 개념입니다. LLM 은 우리 컴퓨터의 함수를 실행할 수 없습니다. 대신 "이 함수를, 이런 인자로 불러줘"라는 요청(JSON) 을 만들어 줄 뿐입니다. 실제 실행은 우리(파이썬) 가 합니다.

### 1. 가장 흔한 오해

많은 사람이 이렇게 상상합니다.

"LLM에게 함수를 주면, LLM이 그 함수를 실행해서 답을 주겠지?"

들렸습니다. LLM은 텍스트(토큰)를 생성하는 기계일 뿐, 여러분의 컴퓨터에서 파이썬 코드를 돌릴 능력이 없습니다. LLM이 하는 일은 딱 하나입니다 — "어떤 함수를 어떤 인자로 부르면 좋겠다"는 결정을 알려 주는 것.

LLM이 만드는 것: `get_stock(product_name="이어버드")` ← 이건 '실행'이 아니라 '요청서'(JSON)  
실제 실행: 우리가 그 요청서를 받아 파이썬으로 함수를 돌림

비유: LLM은 주문서를 쓰는 손님입니다. "아메리카노 한 잔, 샷 추가요"라고 주문서를 작성할 뿐, 직접 커피를 내리지는 않습니다. 커피를 내리는 건 바리스타(우리 파이썬 코드)입니다.

## 2. Function Calling의 5단계

이 "결정 → 실행" 분리를 단계로 풀면 이렇습니다.

1. 우리: 도구 목록(함수+설명)을 모델에 알려줌 + 질문.
2. 모델(LLM): "`get_stock(product_name='이어버드')`를 호출하면 되겠다"고 **JSON**으로 알려줌. (여기까지가 모델의 일 — 모델은 실행을 못 한다!)
3. 우리: 그 함수를 실제로 실행 → "재고 225개" 결과를 얻음.
4. 우리: 결과를 다시 모델에 전달.
5. 모델(LLM): 결과를 보고 사람 말로 최종 답변 작성.

이 5단계가 Function Calling의 전부입니다. 자동이든 수동이든 이 흐름은 똑같습니다 — 다만 누가 3·4단계를 처리하느냐만 다릅니다(다음 절).

## 3. 왜 이렇게 나눠 났을까?

"그냥 LLM이 실행까지 하면 편하잖아?"라고 생각할 수 있습니다. 하지만 이 분리에는 중요한 이유가 있습니다.

- **보안**: LLM이 우리 시스템의 함수를 마음대로 실행하면 위험합니다. "결정만" 받고 실행은 우리가 통제하면, 위험한 호출을 막을 수 있습니다(18강 쓰기 도구).
- **제어**: 실행 전에 인자를 검증하거나, 로그를 남기거나, 권한을 확인할 수 있습니다.
- **유연성**: 같은 "결정"을 받아 다르게 처리할 수 있습니다(재시도, 캐싱 등).

즉, "모델은 결정, 실행은 우리" 라는 구조가 안전하고 유연한 에이전트의 토대입니다.



## 4. 함수의 '설명서'가 핵심

LLM은 함수 본문(코드)을 볼 수 없습니다. 오직 **함수 이름·독스트링·인자 타입**만 보고 "이 도구를 써야겠다"를 판단합니다.

```
def get_stock(product_name: str) -> str:
    """상품명(일부만 입력해도 됨)을 받아 현재 재고 수량을 반환한다."""
    # 위 독스트링이 모델에게 주는 유일한 '사용 설명서'다.
    # product_name: str 처럼 인자 이름과 타입도 모델이 본다.
    ...
```

그래서 독스트링을 **명확히** 쓰는 것이 도구 성능을 좌우합니다. 이걸 6강(도구 설계)에서 깊이 다룹니다.

### 핵심 정리

- LLM 은 함수를 **실행하지 못한다** — "무엇을 부를지" 결정(JSON)만 만든다.
- 실제 실행은 **우리(파이썬)** 가 한다 → "모델은 결정, 실행은 우리".
- 이 분리는 **보안·제어·유연성**을 위한 것이다.
- 모델은 함수의 **이름·독스트링·타입**만 보고 도구를 고른다.

## 자동 실행 vs 수동 루프

Function Calling 의 5 단계(결정→실행→전달→답)에서, **3·4 단계(실행·전달)**를 누가 처리하느냐에 따라 두 방식이 나뉩니다. **자동**은 SDK 가 알아서, **수동**은 우리가 직접 합니다. 자동은 편하고, 수동은 원리를 보여 줍니다.

### 1. 두 방식 비교

| 방식 | 누가 함수를 실행하나        | 언제 쓰나                        |
|----|--------------------|------------------------------|
| 자동 | SDK가 알아서 3·4단계 처리  | 빠르게 결과만 필요할 때                |
| 수동 | 우리가 직접 함수 실행·결과 전달 | 원리 학습, 중간 제어가 필요할 때(→ 7강 루프) |

핵심은 같습니다 — "모델은 결정, 실행은 우리(또는 SDK)". 차이는 그 실행을 자동화하느냐, 손으로 보느냐입니다.

## 2. 자동 실행 — 한 줄로 끝

`config`에 `tools=[함수들]`만 넘기면, SDK가 5단계를 자동으로 돌립니다.

```
from common import get_genai_client, GEMINI_MODEL
from google.genai import types

client = get_genai_client()

resp = client.models.generate_content(
    model=GEMINI_MODEL,
    contents="이어버드 재고 있어?",
    config=types.GenerateContentConfig(
        tools=[get_stock, get_exchange_rate], # ← 도구만 넘기면 끝
        temperature=0,
    ),
)
print(resp.text) # 이미 도구 실행까지 끝난 최종 답변
```

`resp.text`에는 이미 도구가 실행되고 그 결과가 반영된 최종 답이 들어 있습니다. 편하죠. 하지만 중간에 무슨 일이 일어났는지는 보이지 않습니다.

## 3. 수동 루프 — 원리를 눈으로

자동 실행을 끄면(`disable=True`), 모델은 "무엇을 호출할지" 결정만 하고 멈춥니다. 그 결정을 우리가 직접 받아 실행합니다.

```
resp = client.models.generate_content(
    model=GEMINI_MODEL,
    contents="이어버드 재고 알려줘.",
    config=types.GenerateContentConfig(
        tools=[get_stock, get_exchange_rate],
        automatic_function_calling=typesAutomaticFunctionCallingConfig(disable=True), # ← 자동 끄
        temperature=0,
    ),
)

# 모델은 '결정'만 했다 → resp.function_calls 에 담김
for fc in resp.function_calls:
    print("모델의 결정:", fc.name, dict(fc.args)) # 무엇을 부르려는지
    # ↑ 아직 실행 안 됨! 실행은 우리가 한다 (다음 절)
```

`resp.function_calls`에는 모델이 "부르려는 함수 목록"(이름 + 인자)이 들어 있습니다. 아직 실행은 안 됐습니다. 이걸 우리가 직접 실행하는 게 다음 단계입니다.

## 4. 왜 수동을 배우나?

자동이 편한데 왜 수동을 배울까요? 세 가지 이유입니다.

1. **원리 이해**: "모델은 결정만 한다"를 눈으로 봐야 에이전트를 제대로 이해합니다.
2. **중간 제어**: 실행 전 인자 검증, 권한 확인, 로깅을 끼워 넣을 수 있습니다.
3. **7강의 토대**: 7강 Agent Loop는 이 수동 루프를 `while`로 반복하는 것입니다.

정리하면: 자동은 실무에서 빠르게 쓰고, 수동은 원리를 익히고 정교하게 제어할 때 씁니다. 7강에서 이 수동 루프를 반복문으로 감싸 "스스로 여러 단계를 처리하는 에이전트"를 만듭니다.

## 핵심 정리

- 자동 실행 = `tools=[...]`만 주면 SDK가 5 단계를 다 처리(편함).
- 수동 루프 = `disable=True`로 끄고, `function_calls`를 우리가 직접 실행(원리·제어).
- 둘 다 본질은 "모델은 결정, 실행은 우리/SDK".
- 수동 루프는 **7강 Agent Loop의 토대**.

## 따라하기 — 도구로 쓸 함수 만들기

### 목표

LLM에게 주여 줄 도구 두 개(재고 조회·환율 조회)를 파이썬 함수로 만듭니다. 핵심은 독스트링과 타입힌트 — 이게 모델에게 주는 "사용 설명서"입니다. 실제 파일은 `code/ch05_function_calling.py` 입니다.

### 0. 실습 준비 (설치)

```
conda activate agentic

# 5강에 필요한 라이브러리 (앞 강에서 설치했다면 생략 가능)
pip install google-genai python-dotenv pandas
```

이번 강은 `data/exchange_rates.csv` (환율)와 `data/inventory.csv` (재고)를 씁니다.

### 1. 공통 시작부 + 데이터 로드

```
# 실행: python code/ch05_function_calling.py
import sys, pathlib
sys.path.append(str(pathlib.Path(__file__).resolve().parent))
from common import get_genai_client, GEMINI_MODEL, DATA
from google.genai import types
import pandas as pd

client = get_genai_client()
rates = pd.read_csv(DATA / "exchange_rates.csv") # 환율 (currency, krw_per_unit)
inv = pd.read_csv(DATA / "inventory.csv")        # 재고 (product_name, stock, warehouse)
```

## 2. 도구 함수 ① — 환율 조회

```
# [중요] 함수의 독스트링과 타입힌트가 모델에게 주는 '사용 설명서'다(모델은 본문을 못 본다).
def get_exchange_rate(currency: str) -> str:
    """통화 코드(USD, EUR, JPY 등)를 받아 1단위당 원화(KRW) 환율을 반환한다."""
    row = rates[rates["currency"].str.upper() == currency.upper()]
    if row.empty:
        return f"{currency} 환율 정보가 없습니다."
    krw = float(row.iloc[0]["krw_per_unit"])
    return f"1 {currency.upper()} = {krw:,.2f} KRW"
```

뜯어보면:

- 독스트링 `"""통화 코드를 받아..."""` → 모델이 "환율 질문엔 이 도구"라고 판단하는 근거.
- 타입힌트 `currency: str` → 모델이 "문자열 인자를 넣어야 한다"고 이해.
- `-> str` → 결과는 문자열로 돌려준다(모델은 문자열을 읽음).
- 없는 통화면 `"...정보가 없습니다."` 를 돌려줘 빈 결과도 안전하게 처리.

## 3. 도구 함수 ② — 재고 조회

```
def get_stock(product_name: str) -> str:
    """상품명(일부만 입력해도 됨)을 받아 현재 재고 수량을 반환한다."""
    row = inv[inv["product_name"].str.contains(product_name, na=False)]
    if row.empty:
        return f'"{product_name}" 재고 정보를 찾지 못했습니다.'
    r = row.iloc[0]
    return f"{r['product_name']} 재고 {int(r['stock'])}개 ({r['warehouse']})"
```

## 4. 함수가 잘 도는지 먼저 확인 (모델 없이)

도구를 LLM에 붙이기 전에, 함수 자체가 제대로 도는지 직접 호출해 봅니다. (이건 디버깅의 기본입니다.)

```
print(get_stock("이어버드"))      # 승승 무선 블루투스 이어버드 SE 재고 225개 (...)  
print(get_exchange_rate("USD"))   # 1 USD = 1,385.50 KRW
```

### 예상 출력

```
승승 무선 블루투스 이어버드 SE 재고 225개 (서울물류)  
1 USD = 1,385.50 KRW
```

함수가 잘 동작하네요. 이제 이걸 LLM에 도구로 연결할 차례입니다.

## 5. 좋은 도구 함수의 3원칙 (미리보기)

다음 6강에서 자세히 다루지만, 핵심만 미리 봅니다.

1. **독스트링을 명확히**: 모델이 도구를 고르는 유일한 근거.
2. **타입힌트를 정확히**: 모델이 인자를 올바르게 채우게 함.
3. **빈 결과·오류를 안전하게**: 없는 상품·통화면 친절한 메시지 반환(예외로 죽지 않게 — 7강에서 심화).

### 핵심 정리

- 도구 = 독스트링 + 타입힌트를 갖춘 파이썬 함수.
- 독스트링이 모델의 도구 선택 근거이므로 명확히 쓴다.
- 재고처럼 부분 일치が必要하면 str.contains 를 쓰고 독스트링에 명시한다.
- LLM 에 붙이기 전에 함수를 직접 호출해 동작을 확인한다.

## 따라하기 — 자동 Function Calling

### 목표

04번에서 만든 도구 두 개를 LLM에 연결합니다. `tools=[함수들]` 한 줄이면, SDK가 "결정 → 실행 → 결과 전달 → 최종 답"의 5단계를 자동으로 돌립니다.

### 1. 도구를 한 줄로 연결

```
from common import get_genai_client, GEMINI_MODEL
from google.genai import types

client = get_genai_client()
# (get_stock, get_exchange_rate 는 04번에서 만든 함수)

question = "이어버드 재고 있어? 그리고 가격을 달러로 환산하려면 환율이 얼마야?"

resp = client.models.generate_content(
    model=GEMINI_MODEL,
    contents=question,
    config=types.GenerateContentConfig(
        tools=[get_stock, get_exchange_rate], # ← 파이썬 함수를 그대로 도구로!
        temperature=0,
    ),
)
print(resp.text)
```

#### 예상 출력

승승 무선 블루투스 이어버드 SE는 현재 재고 225개가 있습니다(서울물류).  
달러 환산 시 1 USD = 1,385.50 KRW 환율을 적용하시면 됩니다.

## 2. 무슨 일이 일어난 걸까?

우리는 함수 두 개를 `tools`에 넘기고 질문만 했을 뿐입니다. 그런데 LLM이 스스로:

1. 질문을 분석해 "재고 도구 + 환율 도구 둘 다 필요하다"고 판단했고,
2. `get_stock(product_name="이어버드")`와 `get_exchange_rate(currency="USD")`를 호출하기로 결정했고,
3. (SDK가 자동으로) 두 함수를 실행해 결과를 받았고,
4. 그 결과를 종합해 최종 답을 만들었습니다.

우리는 "어떤 도구를 언제 쓰라"고 한마디도 하지 않았습니다. 모델이 알아서 골랐죠. 이게 Function Calling의 힘입니다.

`temperature=0`을 쓴 이유: 도구 선택은 "정답이 정해진" 판단이라, 매번 같은 도구를 고르도록 무작위성을 끕니다(2강·3강 복습).

## 3. 도구가 하나만 필요한 질문은?

질문에 따라 모델은 필요한 도구만 고릅니다.

```
# 재고만 물으면 → get_stock 만 호출
resp = client.models.generate_content(
    model=GEMINI_MODEL,
    contents="로봇청소기 재고 얼마나 있어?",
    config=types.GenerateContentConfig(tools=[get_stock, get_exchange_rate], temperature=0),
)
print(resp.text)
# → "로봇청소기 재고는 ...개 입니다." (환율 도구는 안 씀)
```

모델은 질문을 보고 필요한 만큼만 도구를 씁니다. 환율이 필요 없으면 환율 도구를 부르지 않죠.



## 4. 도구가 없으면?

비교를 위해, 도구 없이 같은 질문을 던져 봅니다.

```
# 도구 없이 — 1강의 그 한계로 돌아감
resp = client.models.generate_content(model=GEMINI_MODEL, contents="이어버드 재고 있어?")
print(resp.text)
# → "실시간 재고는 확인할 수 없습니다 ..." (또는 환각)
```

도구가 있고 없고의 차이가 명확합니다. 도구를 쥐여 주는 순간 LLM이 실제 데이터로 답하게 됩니다.

## 5. 자동의 한계 — 안이 안 보인다

자동은 편하지만, 중간에 무슨 일이 있었는지 보이지 않습니다. "모델이 어떤 도구를 어떤 인자로 불렀나?"를 알 수 없죠. 그걸 들여다보려면 수동 루프가 필요합니다. 다음 절에서 자동 실행을 끄고 모델의 "결정"을 직접 관찰합니다.

### 핵심 정리

- tools=[함수들] 한 줄이면 자동으로 5 단계가 돈다.
- 모델은 질문을 보고 **필요한 도구만 스스로 고른다**.
- 도구 선택은 temperature=0 으로 일관성을 확보한다.
- 자동은 편하지만 **중간 과정이 안 보인다** → 다음 절 수동 루프.

## 따라하기 — 수동 루프로 모델의 결정 관찰

### 목표

자동 실행을 끄고, 모델이 "무엇을 호출하려 했는지"를 직접 들여다봅니다. 그리고 그 결정을 우리가 실행하고, 결과를 모델에 되돌려 최종 답을 만듭니다. 이게 7강 Agent Loop의 토대입니다.

### 1. 자동 실행 끄기

`automatic_function_calling=types.FunctionCallingConfig(disable=True)`로 자동 실행을 끕니다. 그러면 모델은 결정만 하고 멈춥니다.

```
from common import get_genai_client, GEMINI_MODEL
from google.genai import types

client = get_genai_client()

resp = client.models.generate_content(
    model=GEMINI_MODEL,
    contents="이어버드 재고 알려줘.",
    config=types.GenerateContentConfig(
        tools=[get_stock, get_exchange_rate],
        automatic_function_calling=types.FunctionCallingConfig(disable=True), # 자동 끄
        temperature=0,
    ),
)
```

## 2. 모델의 '결정' 들여다보기

이제 `resp.function_calls` 에 모델이 부르려는 함수 목록이 담겨 있습니다. 아직 실행은 안 됐습니다. 우리가 직접 실행합니다.

```
# 이름 → 함수 매핑 (모델이 고른 함수 이름으로 실제 함수를 찾기 위함)
TOOLS = {"get_stock": get_stock, "get_exchange_rate": get_exchange_rate}

for fc in resp.function_calls:
    print("모델의 결정 → 함수:", fc.name, "| 인자:", dict(fc.args))
    result = TOOLS[fc.name](**dict(fc.args))    # ← 우리가 직접 실행!
    print("우리가 실행한 결과:", result)
```

### 예상 출력

모델의 결정 → 함수: `get_stock` | 인자: `{'product_name': '이어버드'}`  
우리가 실행한 결과: 승승 무선 블루투스 이어버드 SE 재고 225개 (서울물류)

여기서 세 가지를 확인하세요.

- `fc.name`: 모델이 고른 함수 이름 (`"get_stock"`)
- `fc.args`: 모델이 채운 인자 (`{'product_name': '이어버드'}`)
- `TOOLS[fc.name](**dict(fc.args))`: 이름으로 진짜 함수를 찾아 우리가 실행

핵심: 모델은 `get_stock` 을 `product_name='이어버드'` 로 부르라고 결정만 했습니다. 실제 실행은 우리 (파이썬)가 했죠. 02번에서 배운 "모델은 결정, 실행은 우리"를 눈으로 확인한 순간입니다.

## 3. `TOOLS` 딕셔너리가 왜 필요한가

모델은 함수 이름(문자열) `"get_stock"` 을 줍니다. 그런데 문자열로는 함수를 호출할 수 없죠. 그래서 이름 → 함수 매핑이 필요합니다.

```
TOOLS = {"get_stock": get_stock, "get_exchange_rate": get_exchange_rate}

name = "get_stock"          # 모델이 준 이름
func = TOOLS[name]          # 이름으로 진짜 함수 객체를 찾음
result = func(product_name="이어버드")  # 호출
```

## 4. 결과를 모델에 되돌려 최종 답 만들기

수동에서 최종 답까지 만들려면, 실행 결과를 `function_response`로 다시 모델에 넣어야 합니다. 대화 기록을 순서대로 쌓습니다.

```
fc = resp.function_calls[0]
result = TOOLS[fc.name](**dict(fc.args))

followup = client.models.generate_content(
    model=GEMINI_MODEL,
    # 대화 기록: [사용자 질문] → [모델의 함수호출] → [우리의 실행 결과] 순으로 쌓는다
    contents=[
        types.Content(role="user",
                       parts=[types.Part(text="이어버드 재고 알려줘.")] ),
        types.Content(role="model",
                       parts=[types.Part(function_call=fc)] ),
        types.Content(role="user",
                       parts=[types.Part.from_function_response(
                           name=fc.name, response={"result": result})]),
    ],
)
print(followup.text)
```

### 예상 출력

승승 무선 블루투스 이어버드 SE는 현재 225개 재고가 있습니다(서울물류 보관).

세 개의 `Content`를 순서대로 쌓은 게 보이시나요?

1. 사용자 질문 ( `role="user"` )
2. 모델의 함수 호출 결정 ( `role="model"` , `function_call=fc` )
3. 우리의 실행 결과 ( `role="user"` , `from_function_response` )

## 5. 자동 vs 수동, 무엇을 봤나

- 자동(05번): `tools=[...]` 만 주면 끝. 편하지만 안이 안 보임.
- 수동(06번): 자동을 끄고 `function_calls`를 직접 실행. 모델의 "결정"이 보이고, 중간에 제어를 끼울 수 있음.

7강에서는 이 수동 루프를 `while`로 감싸, 모델이 "재고 확인 → 부족하면 재주문 기준 조회 → 판단"처럼 여러 단계를 스스로 처리하는 에이전트를 만듭니다.

## 도구 실행 오류 처리

도구는 **항상 성공하지 않습니다**. 없는 상품, 없는 통화, 심지어 DB 다운까지 일어납니다. 오류를 try/except 로 잡아 **친절한 메시지**로 바꿔 모델에 돌려주면, 에이전트가 멈추지 않고 사용자에게 자연스럽게 안내합니다.

### 1. 도구는 왜 실패하는가

실제 운영에서 도구는 다양한 이유로 실패합니다.

- **없는 데이터**: 고객이 "존재하지않는상품XYZ 재고 있어?"라고 물음 → 빈 결과
- **잘못된 입력**: "ZZZ 환율 알려줘" → 지원하지 않는 통화
- **인프라 장애**: 재고 DB가 다운됨 → 연결 실패 예외

이때 도구 함수가 그냥 **예외**를 던지면, 에이전트 전체가 멈춰 버립니다. 고객은 아무 답도 못 받죠. 이건 서비스로서 실격입니다.

비유: 콜센터 상담원(에이전트)이 재고 조회 단말기(도구)가 먹통이 됐다고 **전화를 뚝 끊어 버리면** 안 됩니다. "지금 시스템 점검 중이라 잠시 후 확인 가능합니다"라고 **안내**해야죠.

### 2. 핵심 전략 — 예외를 친절한 문자열로

방법은 간단합니다. 도구 함수 안에서 `try/except` 로 오류를 잡고, **예외 대신 친절한 안내 문자열**을 반환합니다. 그러면 모델이 그 문자열을 보고 사용자에게 안내합니다.

참고: 이건 공식 문서도 권장하는 방식입니다 — *"예외를 던지는 대신, 모델이 처리하도록 둘 수 있다 (instead of raising the exception, you can let the model handle it)".*

```
def get_stock(product_name: str) -> str:
    """상품명을 받아 현재 재고 수량을 반환한다."""
    try:
        row = inv[inv["product_name"].str.contains(product_name, na=False)]
        if row.empty:
            # 빈 결과를 '예외'가 아닌 '친절한 안내'로 바꿔 돌려준다
            return f"'{product_name}' 상품을 찾을 수 없습니다. (상품명을 다시 확인해 주세요)"
        r = row.iloc[0]
        return f"{r['product_name']} 재고 {int(r['stock'])}개 ({r['warehouse']})"
    except Exception as e:
        # 장애는 사용자에게 숨기고, 모델이 안내할 수 있는 메시지로 변환
        return f"일시적인 시스템 오류로 재고를 조회하지 못했습니다. 잠시 후 다시 시도해 주세요. (사유: {e})"
```

두 가지를 처리했습니다.

- 빈 결과(`row.empty`): "상품을 찾을 수 없습니다" 안내
- 예외(`except`): "일시적 오류, 잠시 후 다시" 안내

### 3. DB 다운 시뮬레이션

인프라 장애를 흉내 내기 위해, 스위치 하나로 강제 예외를 던지게 해 봅니다. (실습 파일:

[sections/05\\_function\\_calling/07\\_도구-실행-오류-처리.py](#))

```
DB_DOWN = False # True 면 재고 조회가 강제로 예외를 던진다

def get_stock(product_name: str) -> str:
    """상품명을 받아 현재 재고 수량을 반환한다."""
    try:
        if DB_DOWN:
            raise ConnectionError("재고 DB 연결 실패(timeout)") # 장애 흉내
        # ... (정상 조회 로직)
    except Exception as e:
        return f"일시적인 시스템 오류로 재고를 조회하지 못했습니다. (사유: {e})"
```

## 4. 오류 결과를 모델에 돌려 최종 안내 만들기

06번의 수동 루프 구조를 그대로 쓰되, 도구가 오류 문자열을 반환해도 그걸 모델에 넘깁니다. 모델은 그 오류를 보고 사과하며 안내합니다.

```
def run_turn(question: str) -> str:
    resp = client.models.generate_content(
        model=GEMINI_MODEL, contents=question,
        config=types.GenerateContentConfig(
            tools=[get_stock, get_exchange_rate],
            automatic_function_calling=typesAutomaticFunctionCallingConfig(disable=True),
            temperature=0),
    )
    if not resp.function_calls:
        return resp.text

    history = [types.Content(role="user", parts=[types.Part(text=question)])]
    for fc in resp.function_calls:
        result = TOOLS[fc.name](**dict(fc.args)) # 오류는 함수 내부 try/except 가 흡수
        history.append(types.Content(role="model", parts=[types.Part(function_call=fc)]))
        history.append(types.Content(role="user", parts=[types.Part.from_function_response(
            name=fc.name, response={"result": result})]))

    # 오류 메시지를 포함한 결과를 모델에 되돌려 최종 안내문을 만든다
    followup = client.models.generate_content(
        model=GEMINI_MODEL, contents=history,
        config=types.GenerateContentConfig(
            system_instruction="너는 승승장구물 CS 상담원이다. 도구 결과가 오류면 "
                               "사과하고 다음 행동을 정중히 안내하라.",
            temperature=0.3),
    )
    return followup.text
```

### 예상 출력

[1] 없는 상품 조회

A: 죄송합니다, '존재하지않는상품XYZ' 상품을 찾을 수 없습니다. 정확한 상품명을 알려주시면 다시 확인해 드리겠습니다.

[3] DB 다운 시뮬레이션

A: 죄송합니다. 일시적인 시스템 오류로 재고를 조회하지 못했습니다. 잠시 후 다시 시도해 주시거나, 잠시 뒤 문의

에이전트가 멈추지 않고 정중하게 안내합니다. 오류가 났는데도 서비스가 살아 있죠.

## 5. 왜 이게 중요한가

해피 패스(happy path, 모든 게 잘 되는 경우)만 처리하는 에이전트는 **장난감**입니다. 실제 서비스는 **실패**를 **우아하게 다뤄야** 합니다. 이 "예외를 친절한 메시지로 변환" 패턴은 앞으로 모든 도구·에이전트에서 반복됩니다(12강 풀백, 29강 재시도).

### 핵심 정리

- 도구는 **빈 결과·잘못된 입력·인프라 장애**로 실패할 수 있다.
- try/except 로 잡아 **예외 대신 친절한 문자열**을 반환한다(공식 권장 방식).
- 모델은 그 오류 문자열을 보고 **사과·안내**한다 → 에이전트가 멈추지 않는다.
- 해피 패스만이 아니라 **실패 처리**가 진짜 서비스의 조건이다.



### 3. Tool System

#### 도구가 많아지면 생기는 문제 — "도구 상자가 커지면 헛갈린다"

2 장에서는 도구가 2 개뿐이었습니다. 하지만 진짜 에이전트는 수많은 도구를 갖습니다. 도구가 많아지면 "어떤 도구를 써야 하지?" 라는 새 문제가 생깁니다. 좋은 소식은, 하드코딩 없이 도구 설명(독스트링)만 잘 쓰면 모델이 알아서 고른다는 것입니다.

##### 1. 도구가 늘어나면 생기는 일

5장의 에이전트는 도구가 `get_stock`, `get_exchange_rate` 둘뿐이었습니다. 질문이 오면 둘 중 하나를 고르면 됐죠. 하지만 진짜 승승장구물 CS 에이전트는 이런 도구를 다 가집니다.

- 재고 조회, 가격 조회, 주문 상태 확인, 상품 검색, 환불 정책 안내, 리뷰 요약, 배송 추적...

도구가 10개, 20개로 늘면 새 문제가 생깁니다.

```
# "이 청바지 얼마예요?" 라는 질문에...  
# 모델이 get_order_status(주문 조회)를 부르면? → 엉뚱한 답!  
# 모델이 get_price(가격 조회)를 부르면? → 정답!
```

여러 도구 중 옳은 하나(또는 여럿)를 정확히 고르는 능력이 에이전트 품질을 좌우합니다.

비유: 공구함에 망치 하나만 있으면 고민이 없습니다. 하지만 드라이버·렌치·펜치가 수십 개면, "이 일엔 어떤 공구?" 를 고르는 게 일이 됩니다. 잘못 고르면 나사를 망치로 두드리는 꼴이죠.

## 2. 하드코딩은 답이 아니다

"그럼 우리가 직접 정해 주면 되잖아?"라고 생각할 수 있습니다.

```
# 나쁜 방법 — if문으로 일일이 하드코딩
if "얼마" in question:
    get_price(...)
elif "재고" in question:
    get_stock(...)
elif "주문" in question:
    get_order_status(...)
# ... 도구가 20개면 if문이 20개? 그리고 "값이 어때?"는 "얼마"에 안 걸림!
```

이 방식은 도구가 늘 때마다 `if` 문이 늘고, 표현이 조금만 달라도("얼마" vs "값이 어때?" vs "가격은?") 놓칩니다. 유지보수 지옥입니다.

## 3. 진짜 해법 — 설명을 잘 쓰면 모델이 고른다

좋은 소식이 있습니다. 우리가 도구 선택을 하드코딩할 필요가 전혀 없습니다. 도구의 독스트링(설명)을 잘 써 두면, 모델이 질문과 설명을 비교해 알아서 고릅니다.

```
def get_price(product_name: str) -> str:
    """상품명을 받아 판매가(원)를 반환한다. 가격/얼마 질문에 사용."""
    ...

# 4개 도구를 그냥 다 등록만 하면...
TOOLS = [get_price, get_stock, get_order_status, search_product]

ask("슬림핏 청바지 얼마야?")    # → 모델이 get_price 를 알아서 고름!
ask("이어버드 재고 있어?")     # → 모델이 get_stock 을 알아서 고름!
```

우리는 도구를 그냥 다 등록했을 뿐인데, 질문마다 다른 도구가 선택됩니다. 비결은 독스트링입니다.

### 핵심 정리

- 도구가 많아지면 **"올바른 도구 고르기"**가 새 과제가 된다.
- `if` 문 하드코딩은 유지보수 지옥 → 답이 아니다.
- 독스트링을 잘 쓰면 **모델이 알아서** 도구를 고른다.
- 핵심 한 줄: **"도구 설명은 모델을 위한 프롬프트다."**

## 모델은 무엇을 보고 도구를 고르나

모델은 함수 본문을 보지 못합니다. 오직 함수 이름 + 인자 + 독스트링(설명) 만 봅니다. 이 셋과 사용자 질문을 비교해 가장 잘 맞는 도구를 고릅니다. 그래서 독스트링이 곧 도구 선택의 정확도입니다.

### 1. 모델이 보는 것 vs 못 보는 것

5강에서 배웠듯, 모델은 함수의 코드(본문) 를 절대 못 봅니다. 모델이 보는 건 이것뿐입니다.

```
함수 이름      : get_price
인자          : product_name (문자열)
독스트링(설명): "상품명을 받아 판매가(원)를 반환한다. 가격/얼마 질문에 사용."
```

함수 안에서 `products.csv` 를 읽든 뭘 하든, 모델은 모릅니다. 이름·인자·설명, 이 세 가지가 모델이 가진 정보의 전부입니다.

### 2. 도구 선택은 '매칭'이다

모델은 사용자 질문과 각 도구의 (이름+인자+설명) 을 비교해, 가장 잘 맞는 걸 고릅니다. "청바지 얼마야?"라는 질문이 오면 이런 일이 일어납니다.

질문 "슬림핏 청바지 얼마야?" 를 각 도구 설명과 견줘 봅니다.

| 도구(설명)                                   | 질문과의 매칭              |
|------------------------------------------|----------------------|
| <code>get_stock</code> (재고 수량 반환)        | '얼마'와 안 맞음 X         |
| <code>get_price</code> (판매가 반환, 얼마 질문)   | '얼마 = 가격' 딱 맞음 ✓ 선택! |
| <code>get_order_status</code> (주문 배송 상태) | 주문번호 없음 X            |
| <code>search_product</code> (키워드로 상품 검색) | 특정 상품이 지목됨 X         |

`get_price` 의 독스트링에 "가격/얼마 질문에 사용" 이라고 적어 둔 게 결정적이었습니다. 질문의 "얼마"와 설명의 "얼마"가 매칭된 거죠.

비유: 모델은 도서관 사서입니다. 책 내용(함수 본문)은 안 읽었지만, 책 제목과 요약(이름·설명) 을 보고 "이 질문엔 이 책!"을 골라 줍니다. 그래서 요약을 잘 써 두는 게 중요합니다.

### 3. 코드로 도구 선택 들여다보기

5강의 `which_tool` 처럼, 자동 실행을 끄고 모델이 무엇을 골랐는지 봅니다.

```
from common import get_genai_client, GEMINI_MODEL
from google.genai import types

client = get_genai_client()
TOOLS = [get_price, get_stock, get_order_status, search_product]

def which_tool(question: str):
    """자동 호출을 끄고 모델이 '어떤 도구를 고를지'만 출력한다."""
    resp = client.models.generate_content(
        model=GEMINI_MODEL,
        contents=question,
        config=types.GenerateContentConfig(
            tools=TOOLS,
            automatic_function_calling=typesAutomaticFunctionCallingConfig(disable=True),
            temperature=0,
        ),
    )
    for fc in (resp.function_calls or []):
        print(f" '{question}' → {fc.name}({dict(fc.args)})")

which_tool("슬림핏 청바지 얼마야?")
which_tool("이어버드 재고 있어?")
```

#### 예상 출력

```
'슬림핏 청바지 얼마야?' → get_price({'product_name': '슬림핏 청바지'})
'이어버드 재고 있어?' → get_stock({'product_name': '이어버드'})
```

모델이 질문에 딱 맞는 도구를, 인자까지 정확히 채워 골랐습니다.

## 4. 좋은 독스트링의 두 가지 조건

매칭이 잘 되려면 독스트링이 두 가지를 담아야 합니다.

1. 무엇을 하는가: "상품명을 받아 판매가(원)를 반환한다"
2. 언제 쓰는가: "가격/얼마 질문에 사용"

특히 2번("언제 쓰는가")이 도구 선택에 큰 도움이 됩니다. 모델이 "이 질문이 이 도구를 써야 하는 상황인가?"를 판단하는 힌트가 되니까요.

```
def get_stock(product_name: str) -> str:
    """상품명을 받아 현재 재고 수량과 창고를 반환한다. 재고/품질 질문에 사용."""
    # 독스트링 = '무엇을 하는가'(상품명→재고) + '언제 쓰는가'(재고/품질 질문)
    ...
```

### 핵심 정리

- 모델은 함수 본문을 못 보고, 이름 + 인자 + 독스트링만 본다.
- 도구 선택 = 질문과 도구 설명의 매칭.
- 좋은 독스트링은 "무엇을 하는가" + "언제 쓰는가" 를 담는다.
- which\_tool 로 모델의 선택을 직접 관찰할 수 있다.

### 좋은 도구 설계 원칙

좋은 도구는 다섯 가지 원칙을 지킵니다:

한 도구는 한 가지 일 · 이름이 곧 기능 · 인자가 명확 · 설명이 구체적 · 반환이 일관.  
이 원칙들은 모두 "모델이 도구를 정확히 고르고 올바르게 쓰게" 하기 위한 것입니다.

## 1. 다섯 가지 원칙

| 원칙           | 나쁜 예                                     | 좋은 예                                               |
|--------------|------------------------------------------|----------------------------------------------------|
| 한 도구는 한 가지 일 | <code>do_everything()</code>             | <code>get_price</code> , <code>get_stock</code> 분리 |
| 이름이 곧 기능     | <code>func1</code> , <code>helper</code> | <code>get_order_status</code>                      |
| 인자가 명확       | <code>q: str</code> ("뭐든 넣어")            | <code>product_name: str</code>                     |
| 설명이 구체적      | "정보를 준다"                                 | "상품명을 받아 판매가(원)를 반환한다"                             |
| 반환이 일관       | 때론 dict, 때론 None                         | 항상 사람이 읽는 문자열                                      |

하나씩 봅시다.

## 2. 원칙 ① — 한 도구는 한 가지 일

도구 하나가 여러 일을 하면, 모델은 "이 도구를 언제 써야 하지?"를 판단하기 어렵습니다.

```
# 나쁨 — 한 도구가 가격 · 재고 · 주문을 다 처리
def product_tool(query: str, mode: str) -> str:
    """mode에 따라 가격/재고/주문을 처리한다.""" # 모델이 mode를 잘못 줄 수 있음
    ...

# 좋음 — 일마다 도구를 나눔
def get_price(product_name: str) -> str: ...
def get_stock(product_name: str) -> str: ...
def get_order_status(order_id: str) -> str: ...
```

도구를 나누면 각 도구의 역할이 뚜렷해져, 모델이 고르기 쉽습니다.

## 3. 원칙 ② — 이름이 곧 기능

함수 이름만 봐도 무슨 일을 하는지 알 수 있어야 합니다. 모델은 이름도 선택 근거로 씁니다.

```
def func1(x): ... # 나쁨 — 이름에 정보가 없음
def get_order_status(order_id): ... # 좋음 — 이름이 곧 설명
```

`get_` (조회), `search_` (검색), `update_` (수정) 같은 동사 접두사를 일관되게 쓰면 더 명확합니다.

## 4. 원칙 ③ — 인자가 명확

인자 이름과 타입이 명확해야 모델이 올바른 값을 채웁니다.

```
def lookup(q: str): ...           # 나쁨 — q가 뭘 넣는 건지 모름
def get_price(product_name: str): ... # 좋음 — '상품명'을 넣으라고 명확
def get_order_status(order_id: str): ... # 좋음 — '주문번호'를 넣으라고 명확
```

인자 이름이 `product_name` 이면 모델은 질문에서 상품명을 뽑아 넣고, `order_id` 면 주문번호를 뽑아 넣습니다.

## 5. 원칙 ④ — 설명이 구체적

가장 중요한 원칙입니다(02번에서 강조). 설명은 무엇을 + 언제를 담습니다.

```
def get_price(product_name: str) -> str:
    """상품명(일부만 입력해도 됨)을 받아 판매가(원)를 반환한다. 가격/얼마 질문에 사용."""
    # 독스트링 = '무엇을'(상품명→판매가) + '언제'(가격/얼마 질문)
```

"정보를 반환한다" 같은 모호한 설명은 모델을 헷갈리게 합니다.

## 6. 원칙 ⑤ — 반환이 일관

도구는 항상 같은 형태로 결과를 돌려줘야 합니다. 이 과정에서는 사람이 읽는 문자열로 통일합니다.

```
# 나쁨 — 때론 dict, 때론 None, 때론 숫자
def get_price(name):
    if found: return {"price": 59000} # dict
    else: return None                 # None ← 모델이 처리하기 곤란

# 좋음 — 항상 문자열 (없을 때도 안내 문자열)
def get_price(name):
    if found: return "승승 슬림핏 청바지 판매가 59,000원"
    else: return "'슬림핏 청바지' 가격 정보를 찾지 못했습니다." # 없어도 문자열
```

모델은 문자열을 읽고 답을 만듭니다. 반환이 일관되면 모델이 결과를 안정적으로 다룰 수 있습니다.

## 7. 한 줄로 외우는 원칙

"도구 설명은 모델을 위한 프롬프트다." 동료 개발자가 아니라 모델이 읽는다고 생각하고, 이름·인자·설명·반환을 모두 또렷하게 만드세요.

이 원칙들은 raw SDK든 LangChain이든 똑같이 적용됩니다. "설명을 잘 쓰는 사람"이 좋은 도구를 만듭니다.

### 핵심 정리

- 다섯 원칙: 한 가지 일 · 이름이 곧 기능 · 명확한 인자 · 구체적 설명 · 일관된 반환.
- 모두 "모델이 도구를 잘 고르고 올바르게 쓰게" 하기 위한 것이다.
- 반환은 항상 사람이 읽는 문자열(없을 때도 안내 문자열)로 통일한다.

## 따라하기 — 도구 4개 등록하고 관찰

### 목표

설계 원칙(03번)을 지킨 도구 4개를 만들어 한꺼번에 등록하고, 질문마다 모델이 다른 도구를 고르는 걸 관찰합니다. 실제 파일은 `code/ch06_tools.py` 입니다.

### 0. 실습 준비 (설치)

```
conda activate agentic

# 6강에 필요한 라이브러리 (앞 강에서 설치했다면 생략 가능)
pip install google-genai python-dotenv pandas
```

이번 강은 `data/products.csv`, `data/inventory.csv`, `data/orders.csv`를 씁니다.



## 1. 공통 시작부 + 데이터 로드

```
# 실행: python code/ch06_tools.py
import sys, pathlib
sys.path.append(str(pathlib.Path(__file__).resolve().parent))
from common import get_genai_client, GEMINI_MODEL, DATA
from google.genai import types
import pandas as pd

client = get_genai_client()
products = pd.read_csv(DATA / "products.csv") # 가격 · 카테고리
inventory = pd.read_csv(DATA / "inventory.csv") # 재고 · 창고
orders = pd.read_csv(DATA / "orders.csv") # 주문 · 상태
```

## 2. 도구 4개 정의 (설명을 정성껏)

03번의 5원칙을 지켜 4개를 만듭니다. 각 독스트링이 "무엇을 + 언제" 를 담고 있는 데 주목하세요.

```
def get_price(product_name: str) -> str:
    """상품명(일부만 입력해도 됨)을 받아 판매가(원)를 반환한다. 가격/얼마 질문에 사용."""
    row = products[products["product_name"].str.contains(product_name, na=False)]
    if row.empty:
        return f"'{product_name}' 가격 정보를 찾지 못했습니다."
    r = row.iloc[0]
    return f"{r['product_name']} 판매가 {int(r['price']):,}원"

def get_stock(product_name: str) -> str:
    """상품명(일부만 입력해도 됨)을 받아 현재 재고 수량과 창고를 반환한다. 재고/품절 질문에 사용."""
    row = inventory[inventory["product_name"].str.contains(product_name, na=False)]
    if row.empty:
        return f"'{product_name}' 재고 정보를 찾지 못했습니다."
    r = row.iloc[0]
    return f"{r['product_name']} 재고 {int(r['stock'])}개 ({r['warehouse']})"

def get_order_status(order_id: str) -> str:
    """주문번호(예: 0000106)를 받아 배송 상태를 반환한다. 주문/배송 추적에 사용."""
    row = orders[orders["order_id"] == order_id]
    if row.empty:
        return f"주문번호 {order_id}를 찾지 못했습니다."
    r = row.iloc[0]
    return f"주문 {order_id}: {r['product_name']} {int(r['quantity'])}개, 상태={r['status']}"
```

```
def search_product(keyword: str) -> str:
    """카테고리나 키워드로 상품을 검색해 이름 목록을 반환한다. '어떤 상품 있어?' 류에 사용."""
    hit = products[products["product_name"].str.contains(keyword, na=False) |
                    products["category"].str.contains(keyword, na=False)]
    if hit.empty:
        return f"'{keyword}' 관련 상품이 없습니다."
    return "검색 결과: " + ", ".join(hit["product_name"].head(5).tolist())
```

네 도구가 각각 **가격·재고·주문·검색** 으로 역할이 또렷이 나뉘었습니다(원칙 ① 한 가지 일).

### 3. 4개를 한꺼번에 등록하고 질문하기

```
# 도구 4개를 한 리스트로 묶어 모델에 한꺼번에 등록
TOOLS = [get_price, get_stock, get_order_status, search_product]

def ask(question: str) -> str:
    """질문을 받아 모델이 알맞은 도구를 골라 실행하고(자동), 최종 답변을 돌려준다."""
    resp = client.models.generate_content(
        model=GEMINI_MODEL,
        contents=question,
        config=types.GenerateContentConfig(tools=TOOLS, temperature=0),
    )
    return resp.text

questions = [
    "슬림핏 청바지 얼마야?",          # → get_price 기대
    "이어버드 재고 있어?",           # → get_stock 기대
    "주문 0000106 배송 어디까지 왔어?", # → get_order_status 기대
    "패션의류 쪽에 어떤 상품들 있어?", # → search_product 기대
]

for q in questions:
    print("Q:", q)
    print("A:", ask(q))
    print("-" * 50)
```

## 예상 출력

Q: 슬림핏 청바지 얼마야?

A: 승승 슬림핏 청바지 판매가는 59,000원입니다.

Q: 이어버드 재고 있어?

A: 승승 무선 블루투스 이어버드 SE는 현재 225개 재고가 있습니다(서울물류).

Q: 주문 0000106 배송 어디까지 왔어?

A: 주문 0000106(승승 경량 패딩 점퍼 1개)은 '배송중' 상태입니다.

Q: 패션의류 쪽에 어떤 상품들 있어?

A: 패션의류에는 승승 데일리 코튼 후드티, 승승 경량 패딩 점퍼, 승승 슬림핏 청바지 등이 있습니다.

우리는 4개 도구를 그냥 다 등록했을 뿐인데, 질문마다 다른 도구가 호출됐습니다. 모델이 독스트링을 읽고 매칭한 결과입니다.

## 4. 모델이 고른 도구 들여다보기

`which_tool`로 어떤 도구를 인자까지 어떻게 골랐는지 확인합니다(02번 복습).

```
def which_tool(question: str):
    resp = client.models.generate_content(
        model=GEMINI_MODEL, contents=question,
        config=types.GenerateContentConfig(
            tools=TOOLS,
            automatic_function_calling=typesAutomaticFunctionCallingConfig(disable=True),
            temperature=0),
    )
    for fc in (resp.function_calls or []):
        print(f" '{question}' → {fc.name}({dict(fc.args)})")

for q in questions:
    which_tool(q)
```

## 예상 출력

'슬림핏 청바지 얼마야?' → `get_price({'product_name': '슬림핏 청바지'})`  
'이어버드 재고 있어?' → `get_stock({'product_name': '이어버드'})`  
'주문 0000106 배송 어디까지 왔어?' → `get_order_status({'order_id': '0000106'})`  
'패션의류 쪽에 어떤 상품들 있어?' → `search_product({'keyword': '패션의류'})`

## 핵심 정리

- 도구 여러 개를 TOOLS 리스트에 담아 **한꺼번에 등록**한다.
- 질문마다 모델이 **다른 도구를 스스로** 고른다(독스트링 매칭).
- which\_tool 로 선택한 도구와 **인자**를 확인할 수 있다.

## 따라하기 — 독스트링 망가뜨려 보기

### 목표

독스트링이 정말 중요한지, 일부러 망가뜨려 확인합니다. 모호한 설명을 주면 모델이 도구 선택을 흔들거나 실패하는 걸 직접 봅니다. "독스트링 = 도구 성능"을 체감하는 실험입니다.

### 1. 실험 설계

같은 도구를 두 가지 독스트링으로 준비합니다.

- 자세한 버전: "상품명을 받아 판매가(원)를 반환한다. 가격/얼마 질문에 사용."
- 망가진 버전: "정보를 반환한다."

그리고 같은 질문("청바지 얼마야?")을 두 버전으로 돌려, 모델의 선택이 어떻게 달라지는지 봅니다.

### 2. 코드 — 독스트링만 바꿔 비교

```
from common import get_genai_client, GEMINI_MODEL, DATA
from google.genai import types
import pandas as pd

client = get_genai_client()
products = pd.read_csv(DATA / "products.csv")
```

```

def make_get_price(doc: str):
    """독스트링만 다른 get_price 함수를 만들어 돌려준다."""
    def get_price(product_name: str) -> str:
        row = products[products["product_name"].str.contains(product_name, na=False)]
        if row.empty:
            return f"'{product_name}' 가격 정보를 찾지 못했습니다."
        return f"{row.iloc[0]['product_name']} 판매가 {int(row.iloc[0]['price']):,}원"
    get_price.__doc__ = doc      # 독스트링을 바꿔치기
    return get_price

def get_stock(product_name: str) -> str:
    """상품명을 받아 현재 재고 수량과 창고를 반환한다. 재고/품질 질문에 사용."""
    ... # (생략 — 04번과 동일)

def which_tool(question, tools):
    resp = client.models.generate_content(
        model=GEMINI_MODEL, contents=question,
        config=types.GenerateContentConfig(
            tools=tools,
            automatic_function_calling=typesAutomaticFunctionCallingConfig(disable=True),
            temperature=0),
    )
    calls = resp.function_calls or []
    picked = calls[0].name if calls else "(도구 미선택)"
    print(f" → 모델 선택: {picked}")

```

### 3. 실행 — 자세함 vs 망가짐

```

q = "슬림핏 청바지 얼마야?"

print("[자세한 독스트링]")
good_price = make_get_price("상품명을 받아 판매가(원)를 반환한다. 가격/얼마 질문에 사용.")
which_tool(q, [good_price, get_stock])

print("[망가진 독스트링]")
bad_price = make_get_price("정보를 반환한다.")
which_tool(q, [bad_price, get_stock])

```

## 예상 출력

[자세한 독스트링]

→ 모델 선택: `get_price` ← 정확히 가격 도구를 고름

[망가진 독스트링]

→ 모델 선택: (도구 미선택) 또는 `get_stock` ← 헛갈리거나 엉뚱한 선택

망가진 독스트링 ("**정보를 반환한다**")을 주니, 모델이 "이 도구가 가격 조회용인지" 확신하지 못해 도구를 안 고르거나 엉뚱한 걸 골랐습니다.

## 4. 무엇을 배웠나

이 실험은 한 가지를 분명히 보여 줍니다.

함수 코드는 똑같은데, 독스트링만 바꿨더니 모델의 선택이 달라졌다.

모델은 함수 본문을 못 보고 독스트링만 보기 때문입니다(02번). 그러니 독스트링은 "있으면 좋은 주식"이 아니라 "도구의 성능을 결정하는 핵심 코드"입니다.

## 5. 실무 팁 — 독스트링 점검 체크리스트

도구를 만들 때 독스트링을 이렇게 점검하세요.

- [ ] 무엇을 하는가가 한 문장으로 분명한가? ("판매가를 반환")
- [ ] 언제 쓰는가가 적혀 있는가? ("가격/얼마 질문에 사용")
- [ ] 인자에 무엇을 넣어야 하는지 알 수 있는가? ("상품명")
- [ ] 비슷한 다른 도구와 구별되는가? (다음 절 주제)

마지막 항목 — "비슷한 도구와 구별되는가" — 가 다음 절의 핵심입니다.

## 핵심 정리

- 함수 코드가 같아도 독스트링만 망가지면 도구 선택이 흔들린다.
- 모델은 본문이 아닌 독스트링으로 판단하기 때문이다.
- 독스트링은 "주식"이 아니라 도구 성능을 결정하는 코드다.

## 비슷한 도구 구분하기

설명이 비슷하게 겹치는 두 도구를 두면, 모델이 어느 걸 써야 할지 헷갈립니다.

입력 형태(ID vs 키워드)와 결과 건수(1 건 vs 여러 건)를 독스트링에 **명확히 못 박으면**, 비슷한 도구도 또렷이 구분됩니다.

### 1. 문제 — 헷갈리게 비슷한 두 도구

실무에서는 비슷한 일을 하는 도구가 여럿 생깁니다. 예를 들어 "상품을 찾는" 도구가 두 개 있다고 합시다.

```
def get_product_info(query: str) -> str:
    """상품을 조회한다."""          # ← 모호! 언제 쓰는지 모름
    ...

def search_product(query: str) -> str:
    """상품을 찾는다."""           # ← 위와 거의 같은 설명!
    ...
```

"상품을 조회한다" 와 "상품을 찾는다" — 사람이 봐도 차이를 모르겠죠. 모델은 더 헷갈립니다. "상품 정보 알려줘"라는 질문에 둘 중 뭘 골라야 할지 갈팡질팡합니다.

비유: 회사에 "고객지원팀"과 "고객서비스팀"이 따로 있는데, 둘 다 설명이 "고객을 돕는다"라면? 전화를 어디로 돌려야 할지 아무도 모릅니다. 역할 경계가 모호하기 때문입니다.

### 2. BEFORE — 모호한 설명으로 헷갈리는 모습

(실습 파일: [sections/06\\_tool\\_system/06\\_비슷한-도구-구분하기.py](#))

```
def get_product_info(query: str) -> str:
    """상품을 조회한다."""          # [경고] 모호
    row = products[products["product_id"] == query]
    ...

def search_product(query: str) -> str:
    """상품을 찾는다."""           # [경고] 위와 거의 같은 설명
    hit = products[products["product_name"].str.contains(query, na=False)]
    ...
```

```
# 두 도구로 질문 → 모델이 헛갈림
which_tool("상품ID P0001 상세정보 알려줘.", [get_product_info, search_product])
which_tool("이어버드 들어간 상품들 다 찾아줘.", [get_product_info, search_product])
```

## 예상 출력 (BEFORE)

```
'상품ID P0001 상세정보 알려줘.' → search_product({'query': 'P0001'}) ← 틀린 선택!
'이어버드 들어간 상품들 다 찾아줘.' → get_product_info({'query': '이어버드'}) ← 틀린 선택!
```

둘 다 엉뚱하게 골랐습니다. ID로 1건 찾아야 할 질문에 키워드 검색 도구를, 키워드로 여러 개 찾아야 할 질문에 ID 조회 도구를 골랐죠.

## 3. AFTER — 경계를 못 박은 설명

해결책은 입력 형태와 결과 건수를 설명에 명확히 적는 것입니다.

```
def get_product_info_clear(product_id: str) -> str:
    """[정확검색] 상품ID(예: P0001)로 정확히 1건의 상세정보(이름/카테고리/가격)를 반환한다.
    상품ID를 이미 알 때만 사용. 상품명·키워드로는 사용하지 말 것."""
    ...

def search_product_clear(keyword: str) -> str:
    """[키워드검색] 상품명 일부(예: '이어버드', '청바지')로 여러 후보를 찾아 이름 목록을 반환한다.
    상품ID를 모르고 이름·키워드만 알 때 사용."""
    ...
```

세 가지 장치를 넣었습니다.

- 태그: **[정확검색]** vs **[키워드검색]** — 한눈에 역할 구분
- 입력 형태 명시: "상품ID(P0001)" vs "상품명 일부('이어버드')"
- 언제 쓰지 말지: "상품명·키워드로는 사용하지 말 것" — 경계를 양쪽에서 못 박음

## 예상 출력 (AFTER)

```
'상품ID P0001 상세정보 알려줘.' → get_product_info_clear({'product_id': 'P0001'}) ← 정확!
'이어버드 들어간 상품들 다 찾아줘.' → search_product_clear({'keyword': '이어버드'}) ← 정확!
```

이제 모델이 정확히 구분합니다. 설명만 바꿨을 뿐인데 말이죠.



## 4. 비슷한 도구를 구분하는 3가지 기법

1. 입력 형태를 명시: "ID로" vs "이름 일부로" → 입력이 다르면 도구가 다름이 분명해진다.
2. 결과 건수를 명시: "정확히 1건" vs "여러 후보" → 기대 결과로 구분.
3. 부정 조건을 추가: "~로는 사용하지 말 것" → 잘못된 선택을 막는 울타리.

특히 3번(부정 조건)은 강력합니다. "이 도구를 언제 쓰지 말아야 하는가"를 적으면, 모델이 경계를 더 또렷이 인식합니다.

## 5. 더 나은 해법 — 도구를 합치거나 줄이기

사실 가장 좋은 건, 애초에 헛갈리는 중복 도구를 만들지 않는 것입니다. 두 도구가 정말 비슷하다면:

- 하나로 합칠 수 있는지 (인자로 구분)
- 하나를 없앨 수 있는지

를 먼저 검토하세요. 도구 개수가 적을수록 모델의 선택이 쉬워집니다. 그래도 둘 다 필요하다면, 위의 "경계 못 박기" 기법을 씁니다.

## 핵심 정리

- 설명이 비슷하게 겹치는 도구는 모델을 헛갈리게 한다.
- 해결: 입력 형태 + 결과 건수 + 부정 조건(언제 쓰지 말지) 을 명시한다.
- 더 근본적으로는 중복 도구를 합치거나 줄인다.
- 핵심은 변함없다 — "도구 설명은 모델을 위한 프롬프트다."

## 4. Agent Loop

### 한 번 묻고 끝나지 않는 일

3 장의 도구 호출은 한 방(one-shot)이었습니다 — 질문 → 도구 1 번 → 답.

하지만 현업의 일은 보통 여러 단계입니다.

"행동 → 결과를 보고 → 다음 행동을 다시 정하는" 일을 목표가 끝날 때까지 반복하는 구조가 바로 **Agent Loop** 입니다.

#### 1. one-shot의 한계

지금까지 만든 도구 호출은 한 번에 끝났습니다.

```
ask("이어버드 재고 있어?")  
# → get_stock 1번 호출 → "225개 있습니다" → 끝
```

질문 하나, 도구 하나, 답 하나. 깔끔하죠. 하지만 이런 질문은 어떨까요?

"스마트워치 재고 확인하고, 재고가 부족하면 재주문이 필요한지 알려줘."

이건 한 번에 못 끝납니다. 단계를 밟아야 합니다.

1. 먼저 재고를 조회한다 (행동 1) → "재고 3개"
2. 그 결과를 본다 (관찰) → "어, 3개밖에 없네?"
3. 부족한지 판단하려면 재주문 기준을 확인한다 (행동 2) → "기준은 30개"
4. 비교해 결론을 낸다 (생각) → "3개 < 30개, 재주문 필요!"

행동 1의 결과를 봐야 행동 2를 할지 정할 수 있습니다. 한 방에는 불가능하죠.

#### 2. 그래서 '반복'이 필요하다

핵심은 "행동 → 결과를 보고 → 다음 행동을 다시 정하는" 일을 반복하는 것입니다.

질문: "스마트워치 재고 확인하고 재주문 필요한지 알려줘"

1. 재고 조회 → 관찰: "3개" → 더 알아봐야 함
2. 재주문 기준 조회 → 관찰: "30개" → 이제 판단 가능
3. 판단:  $3 < 30$  → "재주문 필요!" → 최종 답, 종료

이 반복 구조가 **Agent Loop**이고, 에이전트를 진짜 에이전트로 만드는 심장입니다. 1강에서 외운 그 문장 — "LLM이 목표를 보고, 도구를 정해 실행하고, 결과를 관찰해 다음 행동을 다시 정하는 일을 목표가 끝날 때까지 반복" — 이 여기서 코드가 됩니다.

비유: 요리와 같습니다. "국 끓여"라는 목표 하나로 끝나지 않죠. 간을 보고(관찰) → 싱거우면 소금 넣고(행동) → 다시 간을 보고(관찰) → 됐으면 끝. 맛을 봐 가며 조절하는 이 반복이 Agent Loop입니다.

### 3. 이미 배운 것들이 모인다

놀랍게도 Agent Loop는 새로운 게 아니라 지금까지 배운 것의 조합입니다.

| 단계                     | 어디서 배웠나             |
|------------------------|---------------------|
| Thought(생각) — 다음에 뭘 할지 | 4강 CoT (단계적 추론)     |
| Action(행동) — 도구 호출     | 5강 Function Calling |
| 여러 도구 중 선택             | 6강 Tool 시스템         |
| 결정→실행→결과 전달            | 5강 수동 루프            |

7강은 이것들을 **while** 루프로 묶는 강의입니다. 5강의 수동 루프(결정→실행→전달)를 한 번만 하지 않고, 여러 번 반복하는 거죠.

### 4. 이 강에서 만들 것

재고 관리 에이전트를 만듭니다. 도구는 둘로 나눕니다.

- `get_stock(product_name)`: 현재 재고 조회
- `get_reorder_level(product_name)`: 재주문 기준 조회

도구를 일부러 둘로 나눈 이유가 있습니다. 그래야 모델이 "재고 확인 → 기준 확인 → 판단"처럼 두 도구를 차례로 부르게 되어, 다단계 루프가 자연스럽게 만들어집니다. (`data/inventory.csv`의 스마트워치 Fit 5는 재고 3개, 기준 30개라 재주문이 필요한 사례입니다.)

## ReAct 패턴 — 생각.행동.관찰

ReAct 는 Reason(추론) + Act(행동) 의 합성어입니다. 매 스텝마다 모델이 **생각(Thought)** → **행동(Action)** → **관찰(Observation)** 을 거치고, 그 관찰 결과를 다음 입력으로 되돌려 반복합니다. 이 "관찰을 되돌리기"가 ReAct 의 핵심입니다.

### 1. ReAct의 세 박자

매 스텝마다 모델은 세 가지를 거칩니다.

| 단계                      | 하는 일                      | 연관 강   |
|-------------------------|---------------------------|--------|
| <b>Thought (생각)</b>     | "지금 무엇을 해야 하나?"           | 4강 CoT |
| <b>Action (행동)</b>      | "get_stock을 '스마트워치'로 부르자" | 5-6강   |
| (우리가 도구 실행)             | 모델이 정한 도구를 실제로 호출         | —      |
| <b>Observation (관찰)</b> | "재고 3개" — 도구가 돌려준 결과      | —      |

그리고 "충분한가?" 를 판단합니다 — 아니면 다시 **Thought** 로 돌아가 반복하고, 충분하면 **Final Answer**(최종 답변) 를 내놓습니다.

- **Thought:** 모델이 "다음에 뭘 할지" 추론합니다(4강 CoT의 응용).
- **Action:** 어떤 도구를 어떤 인자로 부를지 정합니다(5-6강 Function Calling).
- **Observation:** 우리가 도구를 실행한 결과입니다.

그리고 "충분한가?"를 판단해, 아니면 다시 Thought로 돌아가 반복합니다.

### 2. 핵심은 "관찰을 되돌리기"

ReAct에서 가장 중요한 건, **Observation**(관찰 결과)을 다음 입력으로 다시 모델에 넣는 것입니다.

- 1스텝: 질문 → 모델 → "get\_stock 호출" → 실행 → "재고 3개"
- 2스텝: 질문 + "재고 3개"(관찰 결과를 되돌려 넣음) → 모델 → "재고가 적네, 기준도 봐야겠다" → ...

2스텝에서 모델은 "재고 3개"라는 관찰 결과를 보고 다음 행동을 정합니다. 만약 이 결과를 안 넣어 주면, 모델은 "방금 뭘 봤는지" 기억하지 못해 같은 행동을 반복하거나 엉뚱한 판단을 합니다.

비유: 탐정 수사와 같습니다. 단서 하나(관찰)를 찾으면 그걸 수첩에 적고(되돌리기), 그 단서를 바탕으로 다음 수사 방향(행동)을 정합니다. 단서를 적지 않으면 같은 곳을 계속 뒤지겠죠.

### 3. 왜 도구를 둘로 나눴나

이 강에서는 재고 도구를 둘로 나눕니다: `get_stock` (재고)과 `get_reorder_level` (재주문 기준).

```
def get_stock(product_name: str) -> str:
    """상품명을 받아 현재 재고 수량을 반환한다."""
    ...

def get_reorder_level(product_name: str) -> str:
    """상품명을 받아 재주문 기준 수량을 반환한다. 재고가 이 값 이하이면 재주문이 필요하다."""
    ...
```

만약 두 정보를 한 도구로 묶었다면, 모델은 한 번 호출로 끝내고 루프가 안 돕니다. 일부러 나눠 모델이 "재고 확인 → (관찰) → 기준 확인 → (관찰) → 판단"의 여러 스텝을 밟게 만든 겁니다. 다단계 루프를 연습하기 위한 설계죠.

### 4. ReAct와 4·5·6강의 관계

ReAct는 사실 지금까지 배운 모든 걸 한 루프에 모은 것입니다.

- **Thought** = 4강에서 배운 "단계적으로 생각하기"(CoT)
- **Action** = 5강에서 배운 "도구 호출 결정"
- 도구 선택 = 6강에서 배운 "여러 도구 중 고르기"
- 반복 = 5강 수동 루프를 `while` 로 여러 번

#### 핵심 정리

- ReAct = **생각(Thought) → 행동(Action) → 관찰(Observation)** 의 반복.
- 핵심은 **관찰 결과를 다음 입력으로 되돌리는 것**.
- 도구를 둘로 나눠 모델이 **여러 스텝**을 밟게 설계했다.
- ReAct 는 4 강(생각)·5 강(행동)·6 강(선택)의 종합이다.

## 직접 돌리는 루프와 종료 조건

Agent Loop 는 2 장의 "결정→실행→전달"을 while 로 여러 번 반복하는 것입니다.

단, 반복에는 **종료 조건**이 필수입니다 — (1) 모델이 더 이상 도구를 안 부를 때(정상 종료),  
(2) 최대 반복 횟수(max\_steps, 안전장치) 도달.

### 1. 루프의 뼈대

5장에서 "모델은 결정, 실행은 우리"라고 했습니다. Agent Loop은 그 한 스텝을 `while` 로 감싼 것뿐입니다.

```
while True:
    resp = 모델_호출(대화기록 + 도구목록)
    if resp.function_calls:
        # 모델이 도구를 부르려 한다 (Action)
        결과 = 도구_실행(...)
        # 우리가 실행 (Observation)
        대화기록에 [모델의 호출] + [결과] 추가 # 관찰을 되돌려줌
        continue
        # 다음 스텝으로
    else:
        # 더 부를 도구가 없다 → 최종 답
        print(resp.text)
        break
```

매 바퀴마다 모델은 "도구를 더 부르지" 아니면 "이제 답할지"를 정합니다. 도구를 부르면 실행해서 결과를 되돌리고, 안 부르면 그게 최종 답입니다.

### 2. 종료 조건이 왜 필요한가

`while True` 는 무한 루프의 위험이 있습니다. 만약 모델이 계속 도구만 부르면? 영원히 안 끝나고 토큰(비용)을 끝없이 태웁니다. 그래서 종료 조건을 둘 겁니다.

| 종료 조건        | 의미                          | 종류    |
|--------------|-----------------------------|-------|
| 모델이 도구를 안 부름 | "이제 답할 수 있다"고 판단            | 정상 종료 |
| 최대 반복 횟수 도달  | <code>max_steps</code> 에 도달 | 안전장치  |

### 3. 종료 조건 ① — 정상 종료

모델이 `function_calls`를 비워서 돌려주면, "더 부를 도구가 없다 = 이제 답할 수 있다"는 뜻입니다. 그게 최종 답입니다.

```
if not resp.function_calls:      # 도구를 안 부름
    print("최종 답변:", resp.text) # 이게 답!
    return resp.text
```

예를 들어 재고(3개)와 기준(30개)을 둘 다 확인한 뒤, 모델은 더 이상 도구를 부르지 않고 "재주문이 필요합니다"라고 답합니다. 그게 정상 종료죠.

### 4. 종료 조건 ② — `max_steps` 안전장치

가장 중요한 안전장치입니다. `for step in range(1, max_steps + 1)`로 반복 횟수에 상한을 둡니다.

```
def run_agent(question: str, max_steps: int = 6):
    ...
    for step in range(1, max_steps + 1): # 최대 max_steps 번만 반복
        resp = 모델_호출(...)
        if not resp.function_calls:
            return resp.text          # 정상 종료
        # ... 도구 실행 ...

    print("[종료] 최대 스텝 도달 — 안전장치 작동") # max_steps 초과
    return None
```

`max_steps=6`이면 아무리 꼬여도 6번 안에 멈춥니다. 모델이 버그나 혼란으로 같은 도구를 무한히 부르려 해도, 이 상한이 **비용 폭탄을 막아** 줍니다.

실무에서 `max_steps` (보통 5~10)는 **반드시** 둡니다. 이건 선택이 아니라 필수입니다. "혹시 모를 무한 루프"를 막는 마지막 방어선이거든요. (더 정교한 반복 방지는 06번에서 배웁니다.)

## 5. max\_steps는 몇으로?

- 너무 작으면(예: 1~2): 정상적인 다단계 작업도 끝나기 전에 잘립니다.
- 너무 크면(예: 100): 사고가 나도 한참 토큰을 태운 뒤에야 멈춥니다.
- 적당히(5~10): 대부분의 작업은 몇 스텝이면 끝나므로, 이 정도면 정상 작업은 통과시키고 사고는 막습니다.

작업의 성격에 맞게 정하면 됩니다. 재고 확인 같은 2~3스텝 작업엔 6 정도가 넉넉합니다.

## 핵심 정리

- Agent Loop = 5 강의 "결정→실행→전달"을 while(또는 for)로 반복.
- 종료 조건 둘: 모델이 도구를 안 부름(정상) + max\_steps 도달(안전장치).
- max\_steps 는 무한 루프·비용 폭탄을 막는 필수 안전장치.

## 따라하기 — 미니 에이전트 루프 구현

### 목표

프레임워크 없이 `for` 루프로 ReAct 에이전트를 직접 만듭니다. 실제 파일은 `code/ch07_agent_loop.py` 입니다.

## 0. 실습 준비 (설치)

```
conda activate agentic

# 7강에 필요한 라이브러리 (앞 강에서 설치했다면 생략 가능)
pip install google-genai python-dotenv pandas
```

이번 강은 `data/inventory.csv` (재고 `stock`, 재주문 기준 `reorder_level`)를 씁니다.



## 1. 공통 시작부 + 도구 정의 (둘로 분리)

다단계 루프가 되도록 도구를 둘로 나눕니다(재고 / 재주문 기준).

```
# 실행: python code/ch07_agent_loop.py
import sys, pathlib
sys.path.append(str(pathlib.Path(__file__).resolve().parent))
from common import get_genai_client, GEMINI_MODEL, DATA
from google.genai import types
import pandas as pd

client = get_genai_client()
inv = pd.read_csv(DATA / "inventory.csv")

def get_stock(product_name: str) -> str:
    """상품명(일부만 입력해도 됨)을 받아 현재 재고 수량을 반환한다."""
    row = inv[inv["product_name"].str.contains(product_name, na=False)]
    if row.empty:
        return f'{product_name} 재고 정보 없음'
    r = row.iloc[0]
    return f'{r["product_name"]} 현재 재고 {int(r["stock"])}개'

def get_reorder_level(product_name: str) -> str:
    """상품명을 받아 재주문 기준 수량(reorder_level)을 반환한다. 재고가 이 값 이하이면 재주문이 필요하다."""
    row = inv[inv["product_name"].str.contains(product_name, na=False)]
    if row.empty:
        return f'{product_name} 재주문 기준 정보 없음'
    r = row.iloc[0]
    return f'{r["product_name"]} 재주문 기준 {int(r["reorder_level"])}개'

# 이름 → 함수 매핑 (모델이 고른 이름으로 진짜 함수를 찾기 위함, 5강 복습)
TOOLS = {"get_stock": get_stock, "get_reorder_level": get_reorder_level}
```

## 2. ReAct 루프 본체

이게 이 강의 핵심 코드입니다. 한 줄씩 뜯어봅시다.

```
def run_agent(question: str, max_steps: int = 6):
    """질문을 받아 도구를 반복 호출하며 목표를 끝낼 때까지 도는 ReAct 루프."""
    config = types.GenerateContentConfig(
        tools=list(TOOLS.values()),
        # 자동 실행을 꺼서 '결정 → 우리가 실행 → 결과 전달'을 직접 돌린다
        automatic_function_calling=typesAutomaticFunctionCallingConfig(disable=True),
        system_instruction=(
            "너는 승승장구를 재고 관리 에이전트다. 도구로 재고와 재주문 기준을 확인하고, "
            "재고가 재주문 기준 이하이면 재주문을 권유하라. 한국어로 답하라."
        ),
        temperature=0,
    )
    # 대화 기록(누적): 모델 호출마다 관찰 결과를 덧붙인다 → 모델이 '방금 본 결과'를 근거로 판단
    history = [types.Content(role="user", parts=[types.Part(text=question)])]

    for step in range(1, max_steps + 1):
        resp = client.models.generate_content(
            model=GEMINI_MODEL, contents=history, config=config)

        if not resp.function_calls:
            # 종료 ①: 더 부를 도구 없음 → 최종 답
            print(f"[STEP {step}] 최종 답변")
            return resp.text

        # 모델이 부른 도구(Action)를 history에 기록
        history.append(resp.candidates[0].content)

        # 부른 도구를 우리가 실행(Observation) → history에 결과 추가
        for fc in resp.function_calls:
            print(f"[STEP {step}] 호출:", fc.name, dict(fc.args))
            result = TOOLS[fc.name](**dict(fc.args)) # 우리가 실행
            print("    관찰:", result)
            history.append(types.Content(role="user", parts=[
                types.Part.from_function_response(name=fc.name, response={"result": result})
            ]))

    print("[종료] 최대 스텝 도달 — 안전장치 작동") # 종료 ②: max_steps 초과
    return None
```

### 3. 루프의 세 가지 핵심 장치

이 코드에서 꼭 이해해야 할 세 가지입니다.

#### ① 대화 기록(history)을 누적한다

```
history = [types.Content(role="user", parts=[types.Part(text=question)])]  
# ... 매 스텝마다 history.append(...) 로 모델의 호출 + 우리의 결과를 쌓는다
```

매번 모델을 호출할 때 `history` 전체를 넘깁니다. 그래서 모델이 이전 스텝의 관찰을 보고 판단할 수 있습니다(02번의 "관찰 되돌리기").

#### ② 관찰을 `from_function_response`로 되돌린다

```
history.append(types.Content(role="user", parts=[  
    types.Part.from_function_response(name=fc.name, response={"result": result})  
]))
```

도구 실행 결과를 모델이 읽을 수 있는 형식(`function_response`)으로 만들어 history에 넣습니다. 이게 5강 4-4에서 본 그 패턴입니다.

#### ③ 두 종료 조건

```
if not resp.function_calls: # 종료 ①: 모델이 답하기로 할  
    return resp.text  
# ...  
print("[종료] 최대 스텝 도달") # 종료 ②: for 루프가 max_steps에서 끝남
```

### 핵심 정리

- ReAct 루프 = for step in range(max\_steps) 안에서 모델 호출 → 도구 실행 → 결과 되돌리기.
- 세 핵심: history 누적 · `from_function_response`로 관찰 되돌리기 · 두 종료 조건.
- `disable=True`로 자동 실행을 꺼야 우리가 루프를 제어할 수 있다.

## 따라하기 — 2~3 스텝짜리 일 시키기

### 목표

04번에서 만든 `run_agent`를 실제로 돌려, 모델이 스스로 여러 스텝을 밟는 모습을 관찰합니다. 그리고 같은 루프가 상황에 따라 다른 판단을 내리는 것도 봅니다.

### 1. 재고 부족 → 재주문 권유 (2~3스텝)

스마트워치는 재고가 부족한 사례입니다. 실행해 봅시다.

```
run_agent("스마트워치 재고를 확인하고, 재주문이 필요하면 알려줘.")
```

#### 예상 출력

[STEP 1] 호출: `get_stock` {'product\_name': '스마트워치'}

관찰: 승승 스마트워치 Fit 5 현재 재고 3개

[STEP 2] 호출: `get_reorder_level` {'product\_name': '스마트워치'}

관찰: 승승 스마트워치 Fit 5 재주문 기준 30개

[STEP 3] 최종 답변

Final Answer: 승승 스마트워치 Fit 5 현재 재고는 3개로, 재주문 기준(30개)보다 적습니다. 즉시 재주문을 권장합니다.

### 2. 무슨 일이 일어났나 — 자율성

이 출력에서 가장 중요한 점을 짚어 봅시다.

우리는 "재고부터 보라, 그 다음 기준을 보라"라고 순서를 짜 주지 않았습니다. 그냥 "재고 확인하고 재주문 필요한지 알려줘"라고만 했죠. 그런데 모델은:

1. STEP 1에서 재고를 조회했고,
2. 그 관찰(재고 3개)을 보고 "어, 이게 충분한지 기준을 봐야겠다"고 스스로 판단했고,
3. STEP 2에서 재주문 기준을 조회했고,
4. 둘을 비교해 결론을 냈습니다.

모델이 다음 행동을 스스로 결정했습니다. 이게 1강에서 말한 "AI 에이전트는 AI가 다음 행동을 결정한 다"의 실체입니다. one-shot 도구 호출(5·6강)에서는 볼 수 없던 자율성이죠.

### 3. 재고 충분 → 재주문 불필요 (같은 루프, 다른 판단)

이번엔 재고가 충분한 이어버드로 같은 질문을 해 봅니다.

```
run_agent("이어버드 재고 확인하고 재주문 필요한지 알려줘.")
```

#### 예상 출력

[STEP 1] 호출: `get_stock` {'product\_name': '이어버드'}  
관찰: 승승 무선 블루투스 이어버드 SE 현재 재고 225개

[STEP 2] 호출: `get_reorder_level` {'product\_name': '이어버드'}  
관찰: 승승 무선 블루투스 이어버드 SE 재주문 기준 30개

[STEP 3] 최종 답변

Final Answer: 승승 무선 블루투스 이어버드 SE는 재고가 225개로 재주문 기준(30개)을 크게 웃돕니다. 재주문은 필요하지 않습니다.

똑같은 코드, 똑같은 두 도구인데 결론이 다릅니다. 재고(225) > 기준(30)이므로 "재주문 불필요". 모델이 관찰한 데이터에 따라 판단이 달라진 거죠. 이게 하드코딩된 `if` 문과 다른 점입니다 — 우리는 비교 로직을 짜지 않았는데 모델이 알아서 판단합니다.

### 4. 루프가 흐르는 모습 다시 보기

질문: "스마트워치 재고 확인하고 재주문 필요한지"

- **STEP 1** — 모델 생각: "재고부터 봐야지" → `get_stock` 호출 → 관찰: "재고 3개" (*history에 누적*)
- **STEP 2** — (이전 관찰을 보고) 모델 생각: "3개면 적은데? 기준 확인하자" → `get_reorder_level` 호출 → 관찰: "기준 30개"
- **STEP 3** — 모델 생각: " $3 < 30$ 이니 재주문 필요. 이제 답할 수 있다" → 도구 안 부름 → 최종 답변 → 종료

각 스텝의 관찰이 다음 스텝의 판단 근거가 되는 게 보이시나요? 이게 ReAct 루프의 흐름입니다.

## 무한루프와 반복호출 방지

ReAct 루프는 모델이 같은 도구를 같은 인자로 끝없이 다시 부르며 토큰·비용을 태우는 사고가 날 수 있습니다. `max_steps`(횟수 상한)에 더해, (도구명+인자)를 기록해 중복 호출을 감지하면 "절대 무한루프에 빠지지 않는" 루프가 됩니다.

### 1. 어떤 사고가 나는가

`max_steps` 만으로는 부족할 때가 있습니다. 모델이 진전 없이 같은 행동을 반복하는 경우입니다.

```
STEP 1: get_stock('스마트워치') → "재고 3개"
STEP 2: get_stock('스마트워치') → "재고 3개"   ← 또 같은 호출!
STEP 3: get_stock('스마트워치') → "재고 3개"   ← 또!
... max_steps까지 같은 짓 반복
```

이러면 `max_steps=6` 이라도 6번을 다 채우며 쓸데없이 토큰을 6배 씁니다. 모델이 어떤 이유로 "제자리를 도는" 거죠. 이건 진전이 없으니 빨리 끊어야 합니다.

비유: 길을 잃은 사람이 같은 골목을 빙빙 도는 것과 같습니다. "아까 왔던 길이네?"를 알아채면 멈추고 다른 방법을 찾아야지, 계속 돌면 안 됩니다.

### 2. 두 겹의 안전장치

그래서 안전장치를 두 겹으로 둡니다.

1. `max_steps`: 전체 반복 횟수 상한 (아무리 꼬여도 여기서 멈춤) — 03번에서 배움.
2. 중복 호출 감지: (도구명 + 인자)를 기록해, 같은 호출이 또 나오면 강제 종료.

2번이 새로운 장치입니다. "같은 질문을 같은 도구에 또 던진다 = 진전이 없다"는 신호로 보고 끊습니다.

### 3. 중복 호출 감지 구현

(실습 파일: `sections/07_agent_loop/06_무한루프와-반복호출-방지.py`)

핵심은 이미 실행한 (도구명, 인자) 조합을 `set`에 기억하는 것입니다.

```
def run_agent(question: str, max_steps: int = 6):
    ...
    # 이미 실행한 (도구명, 인자) 조합을 기억하는 집합 — 중복 호출 탐지용
    seen_calls = set()

    for step in range(1, max_steps + 1):
        resp = client.models.generate_content(model=GEMINI_MODEL, contents=history, config=config)

        if not resp.function_calls:
            return resp.text                # 정상 종료

        history.append(resp.candidates[0].content)

        for fc in resp.function_calls:
            # 호출 서명 만들기: args를 정렬된 튜플로 → dict 순서와 무관하게 같은 호출을 같은 키로
            signature = (fc.name, tuple(sorted(dict(fc.args).items())))

            if signature in seen_calls:
                # 같은 도구를 같은 인자로 또 부름 → 진전 없는 반복 → 강제 종료
                print(" [경고] 동일한 도구 · 인자 반복 호출 감지 → 무한루프 방지 강제 종료")
                return None
            seen_calls.add(signature)        # 처음 보는 호출이면 기록

            result = TOOLS[fc.name](**dict(fc.args))
            history.append(types.Content(role="user", parts=[
                types.Part.from_function_response(name=fc.name, response={"result": result})
            ]))

    print("[종료] 단계 초과로 중단 — 최대 스텝 도달(안전장치 작동)")
    return None
```

## 4. '호출 서명'을 만드는 법

핵심 트릭은 호출을 하나의 비교 가능한 키(서명) 로 만드는 것입니다.

```
signature = (fc.name, tuple(sorted(dict(fc.args).items())))  
# (함수명, 인자를 정렬해 만든 튜플) → 같은 호출인지 비교할 지문(signature)
```

- `fc.name`: 함수 이름 ("`get_stock`")
- `tuple(sorted(dict(fc.args).items()))`: 인자를 정렬된 튜플로 변환

인자를 정렬하는 이유는, `{'a': 1, 'b': 2}` 와 `{'b': 2, 'a': 1}` 이 같은 호출인데 dict 순서만 다를 수 있기 때문입니다. 정렬하면 순서와 무관하게 같은 호출을 같은 키로 인식합니다. 이 서명이 `set` 에 이미 있으면 "본 적 있는 호출" = 반복으로 판단합니다.

`set` 을 쓰는 이유: `in` 검사가 매우 빠르고(평균  $O(1)$ ), 중복을 자동으로 걸러 주기 때문입니다.

## 5. 정상 vs 반복 케이스 비교

```
# 정상: 서로 다른 도구를 차례로 → 중복 없이 정상 종료  
run_agent("스마트워치 재고를 확인하고, 재주문이 필요하면 알려줘.")  
  
# 반복 유발: max_steps를 작게(2) 줘서 다 끝나기 전에 안전장치가 작동함을 시연  
run_agent("스마트워치와 이어버드 재고를 모두 확인하고 각각 재주문이 필요한지 알려줘.", max_steps=2)
```

정상 케이스는 `get_stock` → `get_reorder_level` 로 서로 다른 호출이라 중복이 안 걸리고 잘 끝납니다. 반면 작업이 큰데 `max_steps` 가 작으면 안전장치가 먼저 작동해 끝습니다.

## 6. 실무에서의 의미

이 두 안전장치는 실무 에이전트의 기본 골격입니다.

- `max_steps`: 비용 상한선 (절대 넘지 않음)
- 중복 감지: 진전 없는 반복을 조기에 차단

LangChain 같은 프레임워크(8강~)는 이런 안전장치를 내부에 이미 갖추고 있습니다. 우리가 직접 만들어 보면, 프레임워크가 안에서 무슨 일을 하는지 이해하게 됩니다.



## 핵심 정리

- max\_steps 만으로 부족 — 모델이 **같은 호출을 반복**하면 토큰 낭비.
- (도구명+인자) 서명을 **set** 에 기록해 중복 호출을 감지·차단한다.
- 인자는 **정렬된 튜플**로 만들어 순서와 무관하게 같은 호출을 인식한다.
- 두 겹(횟수 상한 + 중복 감지)이면 무한루프에 절대 안 빠진다.

## 대화 기록 누적과 토큰 관리

ReAct 루프는 매 스텝마다 관찰 결과를 history 에 덧붙이므로, **메시지와 토큰이 계속 늘어납니다**. 토큰은 곧 비용·지연이므로, 오래된 기록을 잘라내는 **트리밍(trimming)** 이 필요합니다. 단, **첫 질문(목표)**은 반드시 보존해야 합니다.

### 1. 왜 토큰이 계속 늘어나는가

ReAct 루프는 매 스텝마다 **history** 에 두 개씩 메시지를 추가합니다 — 모델의 도구 호출 + 우리의 실행 결과. 그리고 매번 모델을 호출할 때 **history** 전체를 넘깁니다.

```
STEP 1: history = [질문]                → 토큰 적음
STEP 2: history = [질문, 호출1, 결과1]    → 토큰 늘어남
STEP 3: history = [질문, 호출1, 결과1, 호출2, 결과2] → 더 늘어남
...
```

스텝이 진행될수록 history가 길어지고, **매 호출의 입력 토큰이 선형으로 증가**합니다. 긴 대화나 많은 스텝에서는 이게 **비용 폭탄**이 됩니다.

비유: 회의가 길어질수록 "지금까지 얘기 다시 정리하면..."을 매번 반복하면, 회의가 길어질수록 정리 시간이 점점 더 들죠. history도 마찬가지입니다.

## 2. 토큰 증가를 눈으로 보기

`usage_metadata`로 매 스텝의 토큰을 출력하면 증가가 보입니다. (실습 파일: [sections/07\\_agent\\_loop/07\\_대화 기록-누적과-토큰-관리.py](#))

```
for step in range(1, max_steps + 1):
    resp = client.models.generate_content(model=GEMINI_MODEL, contents=history, config=config)

    um = resp.usage_metadata
    print(f"[STEP {step}] history {len(history)}개 | "
          f"입력 {um.prompt_token_count} + 출력 {um.candidates_token_count} "
          f"= 누적 {um.total_token_count} 토큰")

    if not resp.function_calls:
        break
    # ... history.append(...) 로 기록 추가 ...
```

### 예상 출력

```
[STEP 1] history 1개 | 입력 120 + 출력 18 = 누적 138 토큰
[STEP 2] history 3개 | 입력 185 + 출력 20 = 누적 205 토큰 ← 입력 토큰 늘어남
[STEP 3] history 5개 | 입력 250 + 출력 45 = 누적 295 토큰 ← 더 늘어남
```

`history` 개수가 늘면서 입력 토큰(`prompt_token_count`)이 계속 증가하는 게 보입니다.

## 3. 해결책 — 트리밍(trimming)

오래된 중간 기록은 대개 최종 판단에 덜 중요합니다. 그래서 최근 몇 개만 남기고 잘라냅니다. 단, 한 가지 절대 규칙이 있습니다.

첫 질문(원래 목표)은 반드시 보존한다. 이게 사라지면 모델이 "내가 무슨 일을 하던 중이었지?"를 잊어버립니다.

```
def trim_history(history, keep: int = 4):
    """시스템/첫 사용자 메시지는 보존하고 '최근 keep개'만 유지한다."""
    if len(history) <= keep + 1:
        return history
    head = history[:1]
    tail = history[-keep:]
    return head + tail
```

head(첫 질문) + tail(최근 N 개)를 합쳐 돌려줍니다. 가운데의 오래된 관찰들은 버리는 거죠.

## 4. 트리밍 적용

```
print(f"트리밍 전: {len(history)}개")
trimmed = trim_history(history, keep=2)
print(f"트리밍 후: {len(trimmed)}개 (첫 질문 + 최근 2개 보존)")
# → 다음 호출부터 입력 토큰이 줄어 비용·지연 감소
```

### 예상 출력

```
트리밍 전: 5개
트리밍 후: 3개 (첫 질문 + 최근 2개 보존)
```

5개에서 3개로 줄었습니다. 다음 모델 호출부터는 입력 토큰이 줄어 **비용과 지연이 감소**합니다.

## 5. 트리밍의 트레이드오프

트리밍은 공짜가 아닙니다. 잘라낸 정보는 모델이 못 봅니다.

- 너무 많이 자르면: 모델이 중요한 중간 관찰을 잊어 잘못 판단할 수 있음.
- 너무 적게 자르면: 토큰 절감 효과가 작음.
- 적절히: 최근 몇 스텝 + 첫 질문을 남기는 게 보통 안전.

더 정교한 방법으로는 오래된 대화를 요약해서 압축하는 기법도 있습니다(19강 메모리에서 다룹니다). 트리밍은 "버리기", 요약은 "압축하기"인 셈이죠.

## 6. 비용 감각 정리 (2강 복습)

2강에서 "토큰 = 비용"이라고 배웠습니다. ReAct 루프에서 이게 현실이 됩니다.

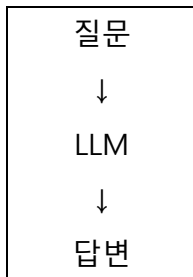
- 스텝이 많을수록 → history가 길어짐 → 입력 토큰 ↑ → 비용 ↑
- 그래서 트리밍(이 절) + 반복 방지(06번) + **max\_steps**(03번)로 비용을 통제
- 이 세 가지가 **비용 효율적인 에이전트**의 기본기

## 5. ReAct(Reason + Act)

**ReAct(Reason + Act)** 는 LLM이 **생각(Reason)** 하고 **행동(Action)** 하며, 행동 결과를 다시 관찰 (Observation)하여 다음 행동을 결정하는 반복적인 추론 방식입니다. 이러한 반복 구조를 **Agent Loop** 라고 합니다.

### 1. ReAct란 무엇인가?

기존 LLM은 대부분 다음과 같이 동작했습니다.



즉, 한 번 질문을 받고 한 번 답변하는 **One-shot 방식**입니다.

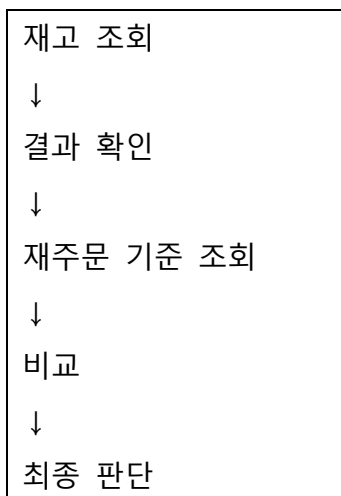
하지만 실제 업무는 대부분 여러 단계를 거쳐야 합니다.

예를 들어,

"스마트워치 재고를 확인하고,  
부족하면 재주문이 필요한지 알려줘."

라는 질문은 한 번에 답할 수 없습니다.

다음과 같은 여러 단계가 필요합니다.



이처럼 여러 단계의 작업을 수행하기 위해 만들어진 방법이 **ReAct** 입니다.

## 2. ReAct의 의미

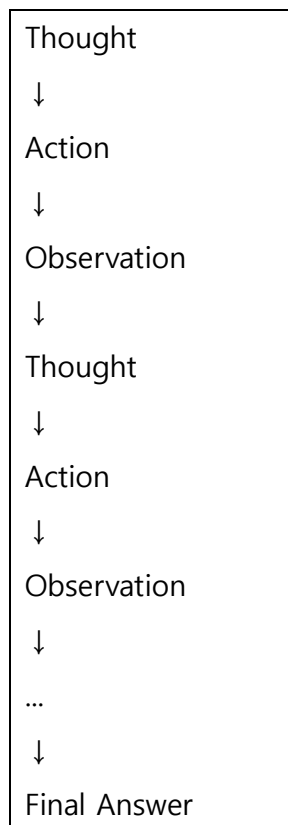
ReAct는 두 단어를 합친 것입니다.

| 단어     | 의미          |
|--------|-------------|
| Reason | 생각한다(추론)    |
| Act    | 행동한다(도구 호출) |

즉, **생각하면서 행동하는 AI** 라는 의미입니다.

## 3. ReAct의 기본 구조

ReAct는 항상 아래 순서로 동작합니다.



즉, **생각 -> 행동 -> 결과관찰 -> 다시 생각을 반복합니다.**

## 4. ReAct의 3단계

### ① Thought (생각)

먼저 AI가 "무엇을 해야 하지?" 를 생각합니다.

예)

재고부터 확인해야겠다.

### ② Action (행동)

생각한 내용을 실제 행동으로 수행합니다.

예)

get\_stock("스마트워치")

즉, Tool Calling이 실행됩니다.

### ③ Observation (관찰)

도구가 실행된 결과를 관찰합니다.

예)

현재 재고 3개

이 결과는 다시 LLM에게 전달됩니다.

Observation

↓

LLM 입력

↓

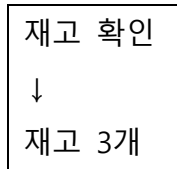
다음 Thought 생성

이 과정이 ReAct의 핵심입니다.

## 5. 왜 Observation이 중요한가?

Observation을 전달하지 않으면 LLM은 방금 무엇을 확인했는지 모르게 됩니다.

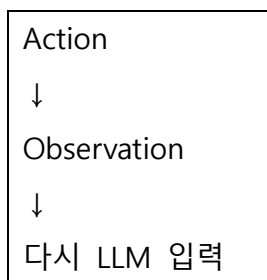
예를 들어



라는 결과를 전달하지 않으면 다음에도 재고 확인을 또 수행할 수 있습니다.

즉, 같은 행동을 반복하게 됩니다.

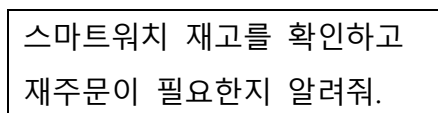
그래서



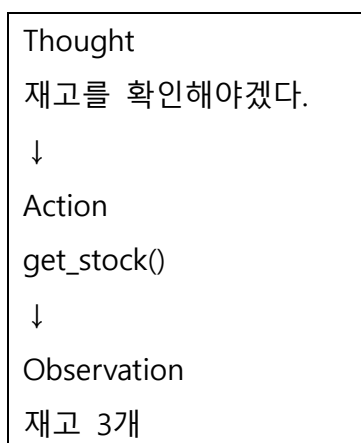
과정이 반드시 필요합니다.

## 6. ReAct 예제

사용자 질문



### STEP 1



## STEP 2

Thought  
3개가 적은지 판단하려면  
재주문 기준을 확인해야겠다.  
↓  
Action  
get\_reorder\_level()  
↓  
Observation  
재주문 기준 30개

## STEP 3

Thought  
 $3 < 30$   
  
재주문이 필요하다.  
↓  
Final Answer  
재주문을 권장합니다.

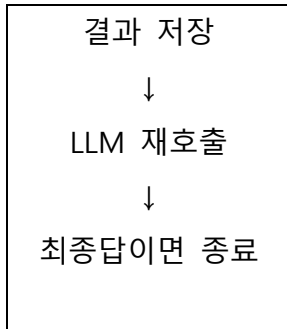
이처럼 여러 단계를 거쳐 최종 답을 생성합니다.

## 7. ReAct와 Agent Loop

ReAct는 실제로 다음과 같은 반복문으로 구현됩니다.

```
while True
    ↓
    LLM 호출
    ↓
    Tool 선택
    ↓
    Tool 실행
```





즉,

LLM



Tool



LLM



Tool



LLM



...

을 반복합니다.

이 구조를 **Agent Loop**라고 합니다.

## 8. 종료 조건

무한 반복을 막기 위해 종료 조건이 필요합니다.

### (1) 모델이 더 이상 Tool을 호출하지 않는 경우

`function_calls == 없음`



최종 답변 출력

## (2) 최대 반복 횟수

max\_steps=6

예)

STEP1

STEP2

STEP3

STEP4

STEP5

STEP6

종료

이렇게 하면 무한루프를 방지할 수 있습니다.

## 9. 무한루프 방지

실제로는 같은 Tool을 계속 호출하는 경우가 있습니다.

예)

get\_stock()

↓

get\_stock()

↓

get\_stock()

↓

get\_stock()

이를 방지하기 위해 (도구명, 인자)를 기록합니다.

예)

|                       |
|-----------------------|
| ("get_stock","스마트워치") |
|-----------------------|

이미 실행된 기록이 있으면

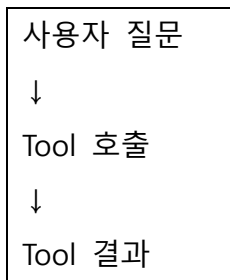
중복 호출

↓

즉시 종료합니다.

## 10. History 관리

매 단계마다



가 계속 저장됩니다.

Question

↓

Call1

↓

Result1

↓

Call2

↓

Result2

↓

Call3

↓

Result3

History가 길어질수록

- 입력 토큰 증가
- 응답 속도 감소
- API 비용 증가

가 발생합니다.

그래서 오래된 History는 잘라내는 **History Trimming** 기법을 사용합니다.

단, 첫 번째 사용자 질문(원래 목표)은 반드시 유지해야 합니다.

## 11. ReAct의 장점

| 장점          | 설명                                 |
|-------------|------------------------------------|
| 다단계 작업 가능   | 여러 Tool을 순차적으로 사용할 수 있음            |
| 추론 능력 향상    | 관찰 결과를 바탕으로 다음 행동 결정               |
| 복잡한 문제 해결   | 단계별 계획을 세워 해결                      |
| 자율성 제공      | AI가 다음 행동을 스스로 결정                  |
| Agent 구현 가능 | Planning Agent, Multi-Agent의 기반 기술 |

## 12. ReAct의 한계

- History가 길어질수록 토큰 사용량이 증가한다.
- 잘못된 Observation이 전달되면 이후 판단도 영향을 받을 수 있다.
- 종료 조건(max\_steps)이 없으면 무한 루프에 빠질 수 있다.
- 동일한 Tool을 반복 호출할 수 있으므로 중복 호출 방지 로직이 필요하다.

## 핵심 정리

- **ReAct = Reason + Act**
- **Thought → Action → Observation → Thought**를 반복하는 추론 패턴이다.
- Observation을 다음 입력으로 되돌려 모델이 이전 결과를 바탕으로 다음 행동을 결정한다.
- ReAct는 **Agent Loop의 핵심 동작 원리**이다.
- 안정적인 구현을 위해서는 **종료 조건(max\_steps), 중복 호출 방지, History Trimming**이 함께 적용되어야 한다.